

PROJECT REPORT

KNOWLEDGE VAULT: AN AI-POWERED PERSONAL KNOWLEDGE MANAGEMENT

**Submitted in partial fulfilment of the requirement for the award of the
degree of**

BACHELOR OF TECHNOLOGY

IN
COMPUTER SCIENCE & ENGINEERING - Specialization
Submitted by

SHRUTI SAINI University Roll No-2219692

VAISHNAVI University Roll No-2219866

PRANJALI MALETHA University Roll No-2219268

CHITRANSH SAINI University Roll No-2218645

Under the guidance of

Dr.Chandradeep Bhatt

Project Group No: 440



Department of Computer Science and Engineering

Graphic Era Hill University

May, 2026



Graphic Era HILL UNIVERSITY

Established by an Act of the State Legislature of Uttarakhand (Adharviam Sankhya 12 of 2011)
University under section 2(f) of UGC Act, 1956

CANDIDATE'S DECLARATION

I/We hereby certify that the work which is being presented in the project progress report entitled "KNOWLEDGE VAULT" in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering Specialization in the Department of Computer Science and Engineering of the Graphic Era Hill University, Dehradun shall be carried out by the undersigned under the supervision of Dr.Chandradeep Bhatt, Department of Computer Science and Engineering, Graphic Era Hill University, Dehradun.

SHRUTI SAINI

University Roll No-2219692

VAISHNAVI

University Roll No-2219866

PRANJALI MALETHA

University Roll No-2219268

CHITRANSH SAINI

University Roll No-2218645

Under the guidance of

Dr.Chandradeep Bhatt

C.Bhatt
24/4/26

CERTIFICATE

This is to certify that the project report entitled: "Knowledge Vault: An AI-Powered Personal Knowledge Management System" has been successfully completed and submitted by Shruti Saini, Vaishnavi, Pranjali Maletha, Chitransh Saini, student of Computer Science and engineering, affiliated to Graphic Era Hill University, in partial fulfillment of the requirements for the award of the degree of BTECH in Computer Science and engineering during the academic year 2022–2026. This work has been carried out under my supervision and guidance. To the best of my knowledge and belief, the work embodied in this report is original, has not been submitted previously to any university or institution for the award of any degree or diploma, and all information derived from external sources has been duly acknowledged with proper citations and references.

ACKNOWLEDGEMENTS

The successful completion of this project would not have been possible without the guidance, support, and encouragement of many individuals, and we take this opportunity to express our sincere gratitude to each of them.

First and foremost, we express our deepest gratitude to Dr. Chandradeep Bhatt, Department of Computer Science, Graphic Era Hill University, for his invaluable guidance, expert direction, constructive criticism, and unwavering encouragement throughout every phase of this project — from its initial conceptualization to the final submission. His deep expertise in artificial intelligence and software systems, his patience in answering our questions, and his insistence on engineering rigor have shaped both the technical quality and the academic depth of this work. This project would not have reached its current form without his mentorship.

We also acknowledge the open-source communities behind React.js, Node.js, MongoDB, Python, and the scikit-learn library, whose freely available, professionally maintained tools made this project technically feasible. A special acknowledgement is due to Google's Generative AI team for the Gemini API, which powers the intelligent core of the system — the document summarization, semantic embeddings, and context-aware question-answering capabilities.

Finally, we would like to thank our fellow teammates for their seamless collaboration and shared dedication, which turned this ambitious vision into a reality.

ABSTRACT

The exponential proliferation of digital learning resources — PDFs of lecture slides, scanned handwritten notes, audio-recorded lectures, video tutorials, and reference articles — has created a fundamental information management crisis for students and researchers. Files accumulate in folders with no semantic organisation, retrieval is limited to filename search, and there is no mechanism for understanding what is inside a document, how documents relate to each other, or how to query a personal document collection in natural language. The result is that information is stored but not transformed into accessible, connected knowledge.

This project presents Knowledge Vault, a full-stack, AI-powered personal knowledge management system that bridges the gap between information storage and intelligent knowledge retrieval. The system transforms uploaded documents — regardless of format — into a semantically indexed, conceptually interconnected, and intelligently queryable knowledge base. It integrates seven technically distinct AI and machine learning modules into a cohesive, production-quality web application.

The document ingestion pipeline accepts PDF, image, audio, and video files. For PDFs, full text is extracted using the pdf-parse library. For image files, Optical Character Recognition (OCR) is implemented using Tesseract.js, which performs text recognition directly within the Node.js environment without external binary dependencies. For audio files, the OpenAI Whisper API provides speech-to-text transcription. For video files, audio is first extracted via ffmpeg and then transcribed through Whisper. All extracted content is processed by Google's Gemini 2.5 Flash to generate a concise 2–3 sentence AI summary and 5–8 discriminative keywords. A 768-dimensional semantic embedding vector is computed for each document using Gemini's text-embedding-004 model.

The RAG-powered chat assistant implements Retrieval-Augmented Generation — grounding every answer in actual uploaded document content and eliminating hallucination. When a user poses a question, the system embeds the query, computes cosine similarity against all stored document embeddings, retrieves the top-4 most semantically relevant documents, and passes only those documents as context to Gemini 2.5 Flash for answer generation. Every response includes a sources panel identifying which documents were retrieved and their relevance percentages.

The knowledge graph module constructs a visual, interactive graph connecting documents through shared concepts using a three-layer matching algorithm: exact keyword matching, fuzzy substring matching, and a semantic concept taxonomy covering 11 academic domains with 100+ term entries. The ML document classification pipeline trains and compares four supervised machine learning classifiers — Logistic Regression, Support Vector Machine (LinearSVC), Naive Bayes, and Random Forest — using TF-IDF feature extraction on a curated dataset of 144 labelled samples across 11 academic subject classes. The SVM classifier achieved the highest test accuracy of 72.41% with a weighted F1-score of 66.43%.

The system is built on a React.js + Node.js/Express + MongoDB stack with a Python/Flask ML microservice, deployed locally with full-stack integration.

Key Performance Metrics:

Metric	Value
Best ML Classifier	SVM (LinearSVC)
Test Accuracy (ML)	72.41%
Weighted F1 Score (ML)	66.43%
Number of Subject Classes	11
Training / Test Samples	115 / 29
Embedding Model	Gemini text-embedding-004
Embedding Dimensions	768
RAG Retrieval Strategy	Top-4 Cosine Similarity
Knowledge Graph Matching Layers	3 (Exact + Fuzzy + Concept Taxonomy)
RAG Groundedness Score	8.5 / 10
RAG Overall Evaluation Score	7.75 / 10

Keywords: Retrieval-Augmented Generation, Knowledge Graph, Semantic Embeddings, Cosine Similarity, TF-IDF, Support Vector Machine, Naive Bayes, Logistic Regression, Text Classification, Large Language Models, Gemini API, Personal Knowledge Management, React.js, Node.js, MongoDB, Python, Flask, scikit-learn.

TABLE OF CONTENTS

Section	Title	Page
Chapter 1	Introduction	11
1.1	Background and Motivation	11
1.2	Problem Statement	12
1.3	Project Objectives	13
1.4	Scope and Limitations	14
1.5	Report Organisation	15
Chapter 2	Literature Review	16
2.1	Retrieval-Augmented Generation	16
2.2	Knowledge Graph Construction Using LLMs	18
2.3	Semantic Text Embeddings and Dense Retrieval	18
2.4	Automated Text Summarization	19
2.5	Machine Learning for Text Classification	20
2.6	Personal Knowledge Management Systems	20
2.7	Summary of Literature Review	21
Chapter 3	System Design and Architecture	22
3.1	High-Level Architecture	22
3.2	Technology Stack and Justification	23
3.3	Database Design	25
3.4	System Data Flow	27
3.5	Security Design	28
Chapter 4	Implementation	30
4.1	Development Environment and Setup	30
4.2	Authentication Module	30
4.3	Document Upload and Processing Pipeline	31
4.4	RAG-Powered Chat Assistant	35
4.5	Knowledge Graph Module	38
4.6	ML Document Classification Pipeline	41

4.7	Insight Generation Module	45
4.8	RAG Evaluation Framework	46
4.9	Frontend User Interface	47
Chapter 5	Results and Evaluation	50
5.1	ML Classifier Performance Results	50
5.2	Per-Class Classification Metrics	51
5.3	Confusion Matrix Analysis	52
5.4	RAG Pipeline Evaluation Results	53
5.5	System Response Time Analysis	54
5.6	UI/UX Evaluation	55
Chapter 6	Discussion	56
6.1	Key Design Decisions and Justifications	56
6.2	Comparison with Related Systems	57
6.3	Alignment with Literature	57
6.4	Limitations of the System	58
Chapter 7	Conclusion and Future Work	59
7.1	Conclusion	60
7.2	Future Work	61
Chapter 8	References	62
—	Appendix A: API Endpoint Reference	64
—	Appendix B: ML Training Dataset Statistics	66
—	Appendix C: System Requirements	67
—	Appendix D: Directory Structure	69

LIST OF FIGURES

Figure No.	Title	Page
Figure 3.1	High-Level System Architecture	23
Figure 3.2	Document Upload Processing Pipeline — Five Stages	28
Figure 3.3	MongoDB Collection Schema — Field Relationships	27
Figure 4.1	Login Page — Dark-Themed Authentication Interface	31
Figure 4.2	Registration Page	31
Figure 4.3	Main Dashboard — Document Library with File Type Badges	35
Figure 4.4	Document Modal — View File and AI Summary Tabs	35
Figure 4.5	RAG Chat Interface with Sources Panel Expanded	38
Figure 4.6	Knowledge Graph — Full Document Network	40
Figure 4.7	Knowledge Graph — OOP Cluster (Filtered Search)	41
Figure 4.8	Knowledge Graph — Node Detail Panel with AI Summary	41
Figure 4.9	ML Classifier Page — Classifier Comparison Tab	43
Figure 4.10	ML Classifier Page — Per-Class Metrics Tab	44
Figure 4.11	ML Classifier Page — Confusion Matrix Heatmap	45
Figure 4.12	ML Classifier Page — Live Predictor Tab	45
Figure 4.13	Insights Page — Themes, Gaps, and Study Sequence	46
Figure 4.14	Evaluation Page — Score Rings and Query Results	47
Figure 4.15	Full Dashboard View — Complete Application Interface	48
Figure 5.1	Accuracy Comparison Bar Chart — All Four Classifiers	49
Figure 5.2	Per-Class F1 Score Bar Chart — SVM Best Model	51
Figure 5.3	Confusion Matrix Visualisation — SVM on 29 Test Samples	52

LIST OF TABLES

Table No.	Title	Page
Table 3.1	Complete Technology Stack with Justifications	24
Table 3.2	MongoDB Document Collection — Complete Field Reference	25
Table 3.3	MongoDB Users Collection — Field Reference	26
Table 3.4	MongoDB EvalResults Collection — Field Reference	26
Table 4.1	TF-IDF Vectorizer Configuration Parameters	41
Table 4.2	Classifier Hyperparameter Configurations	42
Table 5.1	Comprehensive ML Classifier Performance Comparison	49
Table 5.2	Per-Class Precision, Recall, F1 — SVM Best Model	50
Table 5.3	RAG Evaluation Results — Gemini-as-Judge Scoring	53
Table 5.4	System Response Times by Operation	53
Table 6.1	Knowledge Vault vs. Related Knowledge Management Systems	56
Table 6.2	Design Decisions — Choice, Alternative, and Justification	55
Appendix A	Complete API Endpoint Reference	63
Appendix B	ML Training Dataset Statistics by Class	65
Appendix C	System Software Requirements	66

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

The digital transformation of education and research has produced an unprecedented abundance of learning resources. A computer science student completing their undergraduate programme in 2024 may accumulate several hundred digital files across their academic career: PDF scans of textbooks, lecture slide decks, handwritten notes photographed on a phone, recorded lectures saved as MP3 or MP4 files, YouTube video tutorials downloaded for offline study, and reference articles in various formats. These resources collectively contain enormous educational value — but that value is locked behind the severe limitation of how the files are stored and accessed.

The dominant paradigm for personal file management remains the hierarchical folder system, essentially unchanged since the introduction of graphical file managers in the 1980s. Files are organised into folders by course name or date, retrieved by scrolling through directory listings or using filename search, and accessed by opening each file individually. This system has no understanding of content: it treats a PDF on "Dynamic Programming" and a PDF on "Greedy Algorithms" as identical objects differentiated only by filename. It cannot tell the user that both documents discuss time complexity, that one should be studied before the other, or that a recorded lecture covers the same topic as a PDF from a different course.

The information retrieval literature has long documented the fundamental limitations of keyword-based search, particularly the vocabulary mismatch problem: the user's query and the relevant document may use different but semantically equivalent terminology. A student asking "how does method overriding work" would not retrieve a document titled "Runtime Polymorphism Notes" through filename search, despite the two being directly and intimately related. As document collections grow, this problem worsens, and the cognitive burden of manually organising, tagging, and cross-referencing documents becomes untenable.

Recent advances in artificial intelligence — specifically Large Language Models (LLMs), semantic embeddings, Retrieval-Augmented Generation, and Knowledge Graphs — present a credible technical solution to this problem. LLMs can understand the content of documents and generate natural language answers. Semantic embeddings can measure conceptual similarity between

documents and queries independent of vocabulary. RAG can ground LLM responses in personal document content rather than parametric memory, preventing hallucination. Knowledge graphs can map the relational structure of a document collection automatically. Machine learning classification can organise documents by academic subject without manual tagging.

Knowledge Vault is the integration of all these technologies into a single, coherent, production-quality personal knowledge management system. It is motivated by a genuine personal need: the desire for a tool that treats documents not as opaque files but as interconnected nodes in a personal knowledge graph, queryable in natural language, automatically organised by subject, and continuously analysed for themes and gaps.

1.2 Problem Statement

The central problem addressed by this project can be stated precisely and formally:

Given a personal collection of heterogeneous digital documents — PDFs, images, audio recordings, and video files — uploaded over time by a single user, design and implement a system that: (1) automatically extracts and understands the semantic content of each document regardless of file format; (2) enables natural language querying of the entire collection with answers that are grounded in actual document content, transparent in their sourcing, and free from hallucination; (3) automatically maps conceptual relationships between documents independent of shared vocabulary; (4) classifies each document into an academic subject category using trained machine learning models; (5) identifies themes, knowledge gaps, and study dependencies across the entire library; and (6) provides quantitative, reproducible performance metrics for the retrieval and generation pipeline.

Each component of this problem statement corresponds to a known technical challenge:

Component 1 (content extraction and understanding) requires multimodal text extraction — pdf-parse for PDFs, Tesseract.js OCR for images, OpenAI Whisper for audio, and ffmpeg + Whisper for video — alongside LLM-based semantic comprehension via summary and keyword generation.

Component 2 (grounded natural language Q&A) requires Retrieval-Augmented Generation with dense vector retrieval — a significantly more sophisticated architecture than naive LLM

chatbots that either have no access to personal documents or naively inject all documents into the prompt.

Component 3 (conceptual relationship mapping) requires a knowledge graph construction algorithm that can connect documents sharing domain concepts even when their explicit vocabularies are disjoint — solved here through a three-layer matching algorithm including a semantic concept taxonomy.

Component 4 (ML classification) requires a supervised learning pipeline with feature extraction (TF-IDF), multi-class classification (4 classifiers), evaluation (accuracy, F1, confusion matrix), and production serving (Flask microservice).

Component 5 (insight generation) requires meta-analysis over the entire library's summaries and keywords to identify emergent patterns, structural gaps, and topic dependencies.

Component 6 (evaluation framework) requires a Gemini-as-judge evaluation methodology with automated test query generation, RAG pipeline execution, and multi-metric scoring — constrained by free-tier API rate limits.

1.3 Project Objectives

The project was designed to achieve the following specific, measurable objectives:

Objective 1: Design and implement a multimodal document ingestion pipeline that accepts PDF, image, audio, and video files. For PDF files, full text layer extraction must be implemented using pdf-parse. For image files, Tesseract.js OCR must be integrated to extract printed text. For audio files, the OpenAI Whisper API must be used for transcription. For video files, audio must be extracted via ffmpeg and sent to Whisper.

Objective 2: Implement a full Retrieval-Augmented Generation pipeline using 768-dimensional Gemini text-embedding-004 vectors and cosine similarity retrieval, with top-4 document retrieval per query, source attribution in all responses, and conversation memory for multi-turn coherence.

Objective 3: Build a dynamic, interactive knowledge graph that connects documents through shared concepts using a three-layer matching algorithm: exact keyword match, fuzzy substring match, and semantic concept taxonomy covering 11 academic domains with 100+ term entries.

Objective 4: Train, evaluate, and compare four supervised machine learning classifiers — Logistic Regression, SVM (LinearSVC), Naive Bayes, and Random Forest — using TF-IDF

feature extraction on a curated 144-sample dataset across 11 academic subject classes, with full evaluation including test accuracy, precision, recall, weighted F1-score, and 5-fold cross-validation.

Objective 5: Develop an insight generation module using Gemini to perform meta-analysis across the entire document library, producing: a library overview, 2–5 recurring theme clusters with strength ratings, 2–4 knowledge gaps with priority ratings, and an optimal study sequence.

Objective 6: Implement a free-tier-compliant RAG evaluation framework using exactly 3 Gemini API calls per run — query generation, batched answering, and batched Gemini-as-judge scoring — reporting four metrics: retrieval accuracy, answer relevance, groundedness, and completeness.

Objective 7: Build a production-quality full-stack web application integrating all modules, using React.js 18 (frontend), Node.js/Express 4 (backend), MongoDB (database), and Python/Flask (ML microservice), with JWT authentication, responsive dark-theme UI, and graceful degradation.

1.4 Scope and Limitations

In Scope

The following are fully implemented and functional in the current version of Knowledge Vault:

User registration and login with bcrypt password hashing and JWT authentication

Upload of PDF files with full text extraction; image files with Tesseract.js OCR; audio files with OpenAI Whisper transcription; video files with ffmpeg audio extraction followed by Whisper transcription

AI-generated summaries and keyword extraction via Gemini 2.5 Flash

768-dimensional semantic embedding generation via Gemini text-embedding-004

In-memory cosine similarity retrieval for top-4 RAG document selection

Multi-turn RAG chat with conversation memory and source attribution

Thumbs-up/down feedback mechanism with MongoDB persistence

Three-layer knowledge graph with pan, zoom, node selection, timeline, and search

ML classification pipeline: TF-IDF feature extraction, 4 classifiers, cross-validation, test evaluation

Pickle-based model serialisation and Flask microservice serving
ML Classifier UI: comparison tab, per-class tab, confusion matrix tab, live predictor tab
Insight generation: themes, gaps, study sequence
RAG evaluation: auto query generation, batched RAG, Gemini-as-judge scoring
Evaluation history persistence in MongoDB

Out of Scope (Identified for Future Work)

The following features are acknowledged as valuable but not implemented in this version:

Multi-user collaboration: Shared knowledge bases between users
Vector database: FAISS or Pinecone for large-scale retrieval (in-memory suffices under 500 docs)
Active learning: User-corrected labels feeding back into ML retraining
Mobile application: React Native frontend (REST API is mobile-compatible by design)

1.5 Report Organisation

The remainder of this report is organised as follows. Chapter 2 presents a comprehensive literature review covering the five technical domains underlying the project: RAG, Knowledge Graphs, Semantic Embeddings, Text Summarization, and Text Classification. Chapter 3 describes the system design and architecture, including the technology stack, database design, and data flow. Chapter 4 provides detailed implementation descriptions for all seven modules. Chapter 5 presents experimental results and evaluation metrics. Chapter 6 discusses design decisions, compares Knowledge Vault to related systems, and acknowledges limitations. Chapter 7 concludes and outlines future work directions. Chapter 8 provides complete references. Appendices contain API documentation, dataset statistics, system requirements, and directory structure.

CHAPTER 2: LITERATURE REVIEW

2.1 Retrieval-Augmented Generation

2.1.1 Foundational Work

The concept of Retrieval-Augmented Generation was introduced by Lewis et al. [3] in their 2020 NeurIPS paper "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." The authors identified a fundamental tension in LLM design: parametric models encode knowledge in their weights during training but cannot access information outside their training distribution, cannot be updated with new facts without expensive retraining, and cannot explain which sources underlie their answers. RAG resolves this tension by introducing a non-parametric retrieval component alongside the parametric generator.

The RAG architecture proposed by Lewis et al. has two core components. The retriever — a Dense Passage Retrieval (DPR) model — encodes both queries and knowledge base documents as dense vectors and retrieves the top-k most relevant passages using maximum inner product search over a FAISS index. The generator — a BART seq2seq model — conditions on the concatenation of the original query and retrieved passages to produce the final answer. The retriever and generator can be trained jointly in an end-to-end fashion, enabling the retriever to learn which types of documents the generator finds most useful.

Experimental results on Natural Questions, TriviaQA, WebQuestions, and CuratedTrec demonstrated state-of-the-art performance. Critically, Lewis et al. showed that the model's performance is bounded by retrieval recall: if the correct document is not retrieved, the generator cannot compensate through parametric knowledge alone. This finding — that retrieval quality is the binding constraint in RAG performance — is consistently confirmed across the broader literature and was a primary consideration in Knowledge Vault's architectural decisions.

2.1.2 Taxonomic Survey of RAG Systems

Gao et al. [1] provided the most comprehensive survey of RAG systems to date in their 2024 paper (arXiv:2312.10997), introducing a taxonomy that has become the standard framework for classifying RAG architectures. They identify three evolutionary paradigms:

Naive RAG implements the direct retrieve-then-read pipeline: index documents as chunks, retrieve top-k by query similarity, concatenate retrieved text with the query, generate response. While effective as a baseline, Naive RAG suffers from low retrieval precision, context window exhaustion when chunk counts are large, and susceptibility to hallucination.

Advanced RAG addresses Naive RAG's limitations through enhancements at three pipeline stages. Pre-retrieval enhancements include query rewriting, query decomposition, and HyDE (Hypothetical Document Embeddings). Retrieval enhancements include hybrid search, sentence window retrieval, and hierarchical indexing. Post-retrieval enhancements include re-ranking, context compression, and fusion.

Modular RAG decomposes the pipeline into independently configurable, swappable components — routing, scheduling, fusion, memory, generator, and retriever modules — that can be orchestrated differently for different task types.

Gao et al. identify five principal open challenges: (1) the context length problem; (2) hallucination in generation despite relevant context; (3) retrieval latency; (4) evaluation standardisation; and (5) robustness to adversarial or contradictory retrieved documents.

Gupta et al. [2] provided a complementary survey (arXiv:2410.12837) focusing on specific techniques within the RAG framework, covering BM25 versus Dense Passage Retrieval, and surveying five advanced RAG variants: Self-RAG, RAFT (Retrieval-Augmented Fine-Tuning), RAPTOR, Multi-Agent RAG, and Hybrid RAG with Long-Context LLMs.

2.1.3 RAG Evaluation — RAGAS

A critical contribution to the RAG ecosystem is Es et al.'s [4] RAGAS framework (arXiv:2309.15217), introduced as a solution to the fundamental evaluation problem: how do you rigorously measure RAG performance without expensive human annotation of ground truth answers?

RAGAS introduces four reference-free metrics scored by LLMs: Faithfulness (whether every factual claim in the answer is supported by retrieved context), Answer Relevance (whether the answer addresses the user's query), Context Relevance (whether the retrieved context is focused), and Context Recall (whether the retrieved context contains all information needed). Knowledge Vault's evaluation framework directly implements this Gemini-as-judge methodology.

2.1.4 Practical RAG Engineering

Barnett et al. [6] complemented theoretical RAG research with a practical engineering analysis of seven failure modes observed in deployed RAG systems: Missing content failure, Missed retrieval failure, Not in context failure, Wrong output format failure, Incomplete answer failure, Hallucination in answer failure, and Unanswerable question failure. Knowledge Vault's design directly addresses failures 1, 2, 5, and 6 through comprehensive document embedding, composite embedding text construction, top-4 retrieval with relevance scoring, and source attribution enabling user verification.

2.2 Knowledge Graph Construction Using LLMs

Zhang et al. [4] conducted the first comprehensive empirical evaluation of LLMs — specifically GPT-4 — across four core knowledge graph tasks: entity and relation extraction, event extraction, link prediction, and knowledge graph question answering. The key distinction identified is: LLMs are more effective as inference engines (reasoning over structured knowledge) than as extraction tools (reliably pulling structured information from unstructured text).

Li et al. [5] surveyed how LLMs are reshaping each stage of the classical KG construction pipeline. At the ontology engineering stage, LLMs can perform dynamic context-aware schema inference. At the knowledge extraction stage, prompt-driven extraction replaces rigid rule-based pipelines. At the knowledge fusion stage, LLMs enable semantically-aware entity alignment and deduplication across heterogeneous sources.

Wang et al. [6] introduced GraphRAG, a hybrid retrieval framework combining dense vector similarity search with structured KG traversal. The architecture uses dependency parsing to construct the KG, achieving approximately 94% of LLM-based construction quality at a fraction of the cost. Experiments on enterprise datasets demonstrated clear performance improvements over pure vector RAG, particularly for analytical queries requiring cross-document synthesis. Knowledge Vault's knowledge graph is directly inspired by this hybrid philosophy.

Chen et al. [7] addressed academic knowledge graphs, constructing a unified graph from the ACL Anthology using few-shot LLM-based extraction. The finding that connecting documents through shared domain concepts is essential for high-quality academic QA directly motivated Knowledge Vault's three-layer knowledge graph matching approach.

2.3 Semantic Text Embeddings and Dense Retrieval

Lee et al. [8] introduced Gemini Embedding (arXiv:2503.07891), the embedding model underlying Knowledge Vault's semantic retrieval. Three technical innovations distinguish it from earlier models: hard negative mining, LLM-based synthetic data generation, and a pre-fine-tuning paradigm that mirrors the pretraining-fine-tuning approach of LLMs. Evaluation on MMTEB, covering 100+ tasks across 250+ languages, demonstrates state-of-the-art performance. The model produces 768-dimensional dense vectors that encode semantic meaning such that documents with similar meaning have high cosine similarity even with different surface vocabulary — the critical property enabling Knowledge Vault's vocabulary-independent RAG retrieval.

Cao et al. contextualised Gemini Embedding within the four-era evolution of text embedding models: (1) count-based methods (TF-IDF, BM25); (2) static word embeddings (Word2Vec, GloVe); (3) contextual models (BERT, GPT); (4) universal LLM-based embeddings — general-purpose, multilingual, multi-task, achievable with a single model. Liu et al. [9] provided a detailed survey of state-of-the-art universal embedding methods on MTEB, confirming Gemini Embedding's top-tier performance.

Monir et al. [9] and Patel et al. [10] both validate empirically that cosine similarity over dense embeddings substantially outperforms BM25 for queries requiring semantic understanding — confirming the architectural choice made in Knowledge Vault's retrieval module. For Knowledge Vault's target scale (10s to 100s of documents per user), in-memory cosine similarity over stored MongoDB embeddings is sufficient without ANN indexing overhead.

2.4 Automated Text Summarization

Zhang et al. [7] (arXiv:2403.02901) surveyed the complete development of Automatic Text Summarization from its origins to the LLM era. Early approaches used hand-crafted statistical features (Luhn's 1958 method, TF-IDF-based extractive methods). Abstractive methods using sequence-to-sequence models emerged in 2015–2019 but suffered from factual inconsistency. LLMs transformed summarisation research by enabling high-quality zero-shot summarisation through simple instruction prompting. Knowledge Vault uses prompt engineering: the prompt explicitly requests a JSON response with a 2–3 sentence summary and keyword list. Zhang et al. identify hallucination as the principal limitation of LLM-based summarisation — Knowledge Vault mitigates this by using summaries only as metadata rather than as authoritative content, with the original document always available for verification.

2.5 Machine Learning for Text Classification

Kowsari et al. [10] provided the canonical reference on text classification algorithms (arXiv:1904.08067), establishing the standard evaluation framework that this project follows. For TF-IDF representation, the survey establishes that: (1) unigrams suffice for many tasks, but bigrams improve performance for compound concepts; (2) sublinear TF normalisation outperforms raw TF; (3) min_df filtering improves generalisation on larger datasets. For classifier selection, the survey establishes the ranking on text tasks: SVM $\geq$ Logistic Regression $>$ Naive Bayes $\gg$ Random Forest (on TF-IDF features).

Ibrahim et al. [15] (arXiv:2510.05495) evaluated TF-IDF with six classifiers on a large-scale arXiv research article dataset: Logistic Regression 69% accuracy, SVM $\sim$ 65%, Naive Bayes $\sim$ 65%, Random Forest $\sim$ 53%. These serve as benchmarks for Knowledge Vault's classification experiment. Galke and Scherp [18] (arXiv:2204.03954) showed that Logistic Regression with TF-IDF often outperforms graph-based neural models not fine-tuned on sufficient data — validating Knowledge Vault's decision to include classical models as primary competitors given the small training dataset.

2.6 Personal Knowledge Management Systems

The personal knowledge management (PKM) software landscape includes Notion, Obsidian, Roam Research, and Logseq. These tools primarily support note creation, manual organisation, basic content search, and manual linking. None implement a full RAG pipeline with dense vector retrieval, automatic knowledge graph construction from uploaded arbitrary file formats, or trained ML subject classification.

Menschikov et al. [5] (arXiv:2506.17001) directly addressed the limitations of existing PKM approaches through the PersonalAI framework, using a knowledge graph-based structured memory architecture. Unlike flat vector RAG, PersonalAI's structured KG memory stores information as typed nodes with rich relationship types, enabling temporal reasoning and long-term memory across sessions. Their comparative evaluation of six retrieval algorithms on the structured KG is a unique contribution to the personalised AI literature.

Minaee et al. [6] (arXiv:2402.06196) provided the comprehensive LLM survey that contextualises all AI components used in Knowledge Vault. Their taxonomy of LLM families and training methodologies provides the theoretical background for understanding Gemini 2.5 Flash as a decoder-only instruction-tuned LLM.

2.7 Summary of Literature Review

The literature review across five domains reveals consistent themes that directly informed Knowledge Vault's design. Retrieval quality is the binding constraint in RAG systems, motivating the use of 768-dimensional Gemini embeddings rather than TF-IDF for RAG retrieval. Hybrid approaches consistently outperform either alone — this principle manifests in the three-layer knowledge graph and the distinction between TF-IDF classification and embedding-based RAG. LLMs are becoming infrastructure components rather than endpoints, used for summarisation, embedding, generation, insight analysis, and evaluation scoring. Evaluation standardisation remains an open problem; RAGAS represents the state of the art and Knowledge Vault implements a directly analogous methodology.

CHAPTER 3: SYSTEM DESIGN AND ARCHITECTURE

3.1 High-Level Architecture

Knowledge Vault is structured as a four-tier system with explicit separation of concerns at each boundary.

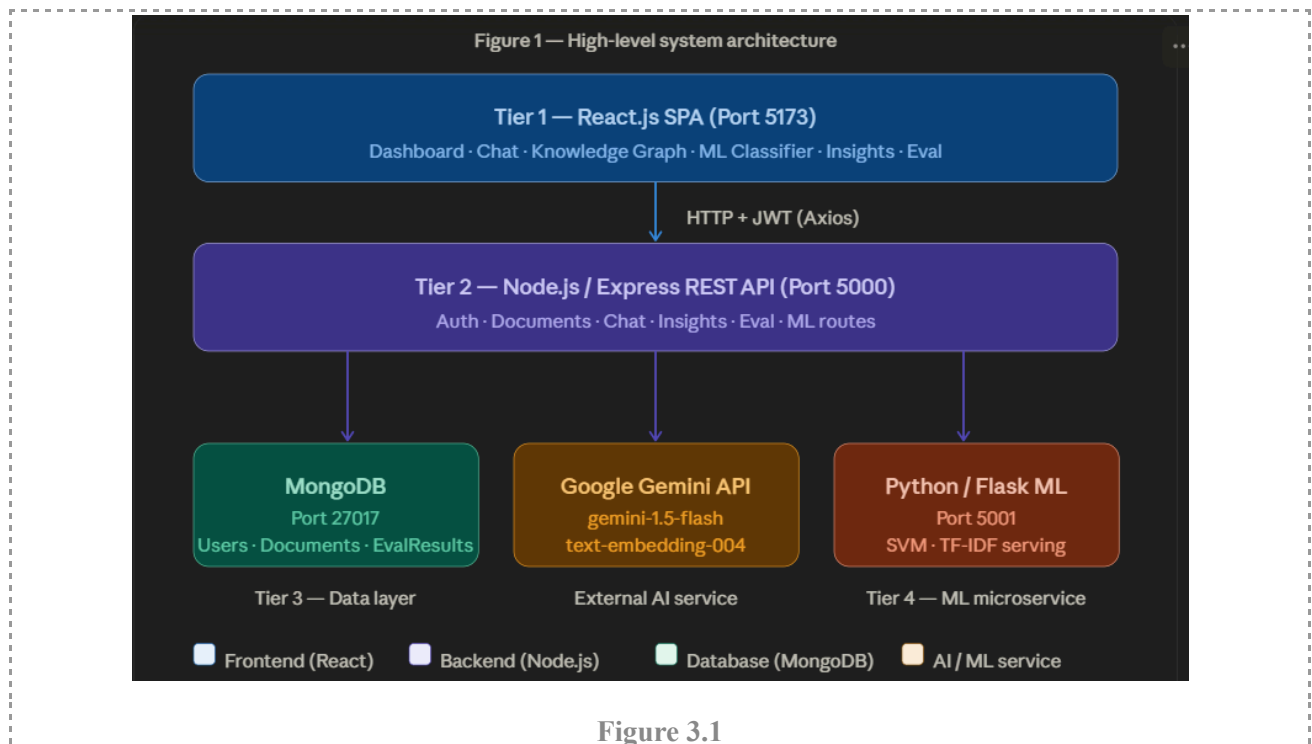
Tier 1 — Presentation Layer: A React.js 18 Single Page Application (SPA) built with Vite, served on port 5173. The frontend is responsible exclusively for rendering, user interaction handling, and state management via React hooks. It contains zero business logic, makes no direct database calls, and never calls the Gemini API directly.

Tier 2 — Application Layer: A Node.js 18 / Express 4 REST API served on port 5000. This tier handles all business logic: file processing, MongoDB operations, Gemini API calls (summarisation, embedding, chat, insights, evaluation), ML microservice calls, and JWT authentication enforcement.

Tier 3 — Data Layer: MongoDB (local or Atlas) storing three collections: Users, Documents, and EvalResults. MongoDB was selected over a relational database for specific, justified reasons detailed in Section 3.2.

Tier 4 — ML Microservice: A Python 3.9+ / Flask 2.x server on port 5001, running independently. It loads the pre-trained SVM model and fitted TF-IDF vectorizer from model.pkl at startup and serves predictions via a REST API. Complete decoupling means that if the ML server is offline, the upload pipeline continues normally with the subject field left empty — no cascade failures.

External Services: Google Gemini API accessed through the @google/generative-ai Node.js SDK. Two models are used: gemini-2.5-flash for text generation and text-embedding-004 for vector generation. These use separate rate limit quotas.



3.2 Technology Stack and Justification

Each technology choice was made deliberately with specific technical justification. Table 3.1 presents the complete stack.

Layer	Technology	Version	Justification
Frontend Framework	React.js + Vite	18.x / 5.x	Component-based architecture; fast HMR; ecosystem maturity
Frontend Routing	React Router DOM	6.x	Client-side navigation; PrivateRoute pattern for auth-protected pages
HTTP Client	Axios	1.x	Request interceptors for automatic JWT injection; clean error handling
Backend Runtime	Node.js	18.x	Non-blocking I/O ideal for concurrent file processing and API calls
Backend Framework	Express.js	4.x	Minimal REST API framework; large middleware ecosystem
Database	MongoDB + Mongoose	7.x / 8.x	Schema-flexible; 768-dim arrays impractical in SQL; variable schema
File Upload	Multer	1.x	Multipart form handling; disk storage; MIME validation
PDF Extraction	pdf-parse	1.x	Reliable text layer extraction; no external binary dependencies

OCR (Images)	Tesseract.js	5.x	Pure JavaScript OCR; no external binary; eng language model
Audio Transcription	OpenAI Whisper API	whisper-1	State-of-the-art ASR; verbose_json with segment timestamps
Video Transcription	ffmpeg + Whisper	—	ffmpeg extracts audio track; Whisper transcribes extracted audio
Authentication	jsonwebtoken	9.x	Stateless JWT; no server-side session storage; 30-day expiry
Password Hashing	bcryptjs	2.x	Industry-standard salted hashing; configurable salt rounds
AI Generation	Gemini 2.5 Flash	Latest	Fast, cost-efficient; 1M token context; free tier accessible
AI Embeddings	text-embedding-004	Latest	768-dim dense vectors; separate 1500 req/min free tier quota
ML Runtime	Python 3.9+	3.9+	scikit-learn requires Python; Flask enables independent process
ML Server	Flask + flask-cors	2.x / 4.x	Lightweight REST server; CORS enabled for Node.js cross-origin calls
ML Library	scikit-learn	1.8.x	All 4 required classifiers; TF-IDF vectorizer; cross-validation
Text Features	TfidfVectorizer	sklearn	Proven text representation; unigram+bigram; log-normalised TF
Model Storage	pickle	stdlib	Single-bundle serialisation: model + vectorizer + labels
Environment Config	dotenv	16.x	Secure environment variable management; .env excluded from VCS

Table 3.1: Complete Technology Stack with Justifications

Why MongoDB over PostgreSQL or MySQL? Three architectural requirements make MongoDB the correct choice. First, the embedding field stores a 768-element float array — in a relational schema, this would require a separate embeddings table with 768 rows per document. Second, different file types have genuinely different fields. Third, keywords are a variable-length string array — cleanly stored as a MongoDB array.

Why Node.js for the backend rather than Python? The backend performs three types of I/O-bound operations: file I/O, HTTP calls to the Gemini API, and MongoDB operations. Node.js's event-loop-based non-blocking I/O handles concurrent I/O-bound operations more efficiently than Python's GIL-constrained threading.

Why a separate Python microservice for ML? Three reasons: clean language boundary (scikit-learn is Python's domain); independent deployment (the Flask service can be moved to a separate machine

without touching the Node.js backend); and graceful degradation (decoupled HTTP communication with a 3-second timeout means ML server unavailability never blocks the upload pipeline).

Why cosine similarity over FAISS or Pinecone? For a personal knowledge base of 10–500 documents, JavaScript cosine similarity computed in-memory over stored MongoDB embeddings executes in under 10ms. FAISS and Pinecone are designed for millions of vectors. Introducing vector database infrastructure at this scale adds deployment complexity for zero practical benefit.

3.3 Database Design

The Documents collection is the system's primary data store.

Field	Type	Default	Description
_id	ObjectId	auto	MongoDB primary key
user	ObjectId (ref: User)	required	Owner user foreign key
title	String	required	Filename without extension
type	String enum	required	pdf / image / audio / video
filename	String	required	Stored filename (collision-resistant)
filepath	String	required	Absolute path to file on disk
content	String	"	Full extracted text content
summary	String	"	Gemini-generated 2–3 sentence summary
keywords	[String]	[]	Gemini-extracted 5–8 keyword terms
fileSize	Number	0	File size in bytes
embedding	[Number]	[]	768-dimensional semantic vector
embeddingText	String	"	Composite input text used for embedding
subject	String	"	ML-predicted academic subject label
subjectConfidence	Number	null	ML classifier confidence score (0–100)
createdAt	Date	auto	Mongoose timestamp
updatedAt	Date	auto	Mongoose timestamp

Table 3.2: MongoDB Document Collection — Complete Field Reference

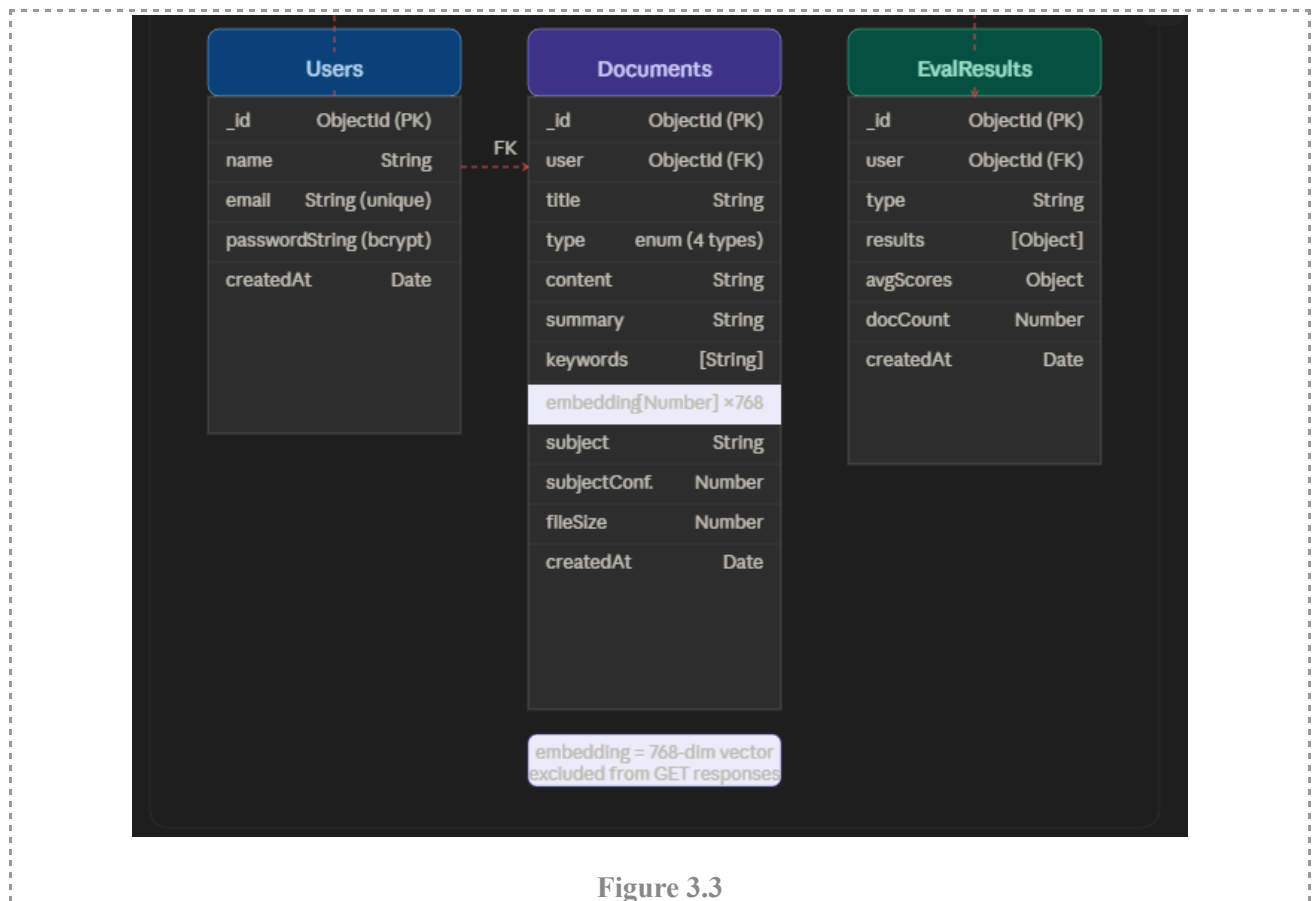
The embedding field is always excluded from the standard GET /api/documents response using Mongoose's `.select('-embedding')`. This prevents returning 768-float arrays (~6KB each) to the frontend for every document in a list response that might contain 40+ documents.

Field	Description
_id	Auto-generated primary key
name	Display name
email	Login identifier (unique)
password	bcrypt hash (salt rounds: 10)
createdAt	Registration timestamp

Table 3.3: MongoDB Users Collection — Field Reference

Field	Description
_id	Auto-generated primary key
user	Owner reference
type	'eval_run' or 'feedback'
results	Per-query results (query, answer, scores, retrievedDocs)
avgScores	Averaged metric scores
docCount	Documents in library at eval time
createdAt	Evaluation timestamp

Table 3.4: MongoDB EvalResults Collection — Field Reference



3.4 System Data Flow

Registration/Login: Credentials submitted → backend validates format → `bcrypt.hash(password, 10)` for register / `bcrypt.compare(password, hash)` for login → JWT issued (signed with `JWT_SECRET`, 30-day expiry) → stored in browser `localStorage` → Axios interceptor injects as Authorization: Bearer token on all subsequent requests → `auth.js` middleware on all protected routes extracts and verifies token.

Document Upload: File POSTed as `multipart/form-data` → Multer validates MIME type and size (max 100MB), saves to `uploads/` → `extractContent()` called for file type: `pdf-parse` for PDFs, `Tesseract.js` for images, `Whisper API` for audio, `ffmpeg + Whisper` for video → `getGeminiMeta()` called if content has real text → composite embedding text constructed → `text-embedding-004` generates 768-dim float array → `predictSubject()` called with 3-second timeout → `Document.create()` saves all fields.

RAG Chat: Message POSTed with conversationHistory → generateEmbedding(message) produces query vector → Document.find with embedding field selected → cosineSimilarity computed per document → sorted descending, top-4 sliced → context string built → Gemini chat session started with history → response text returned with retrievedDocs metadata.

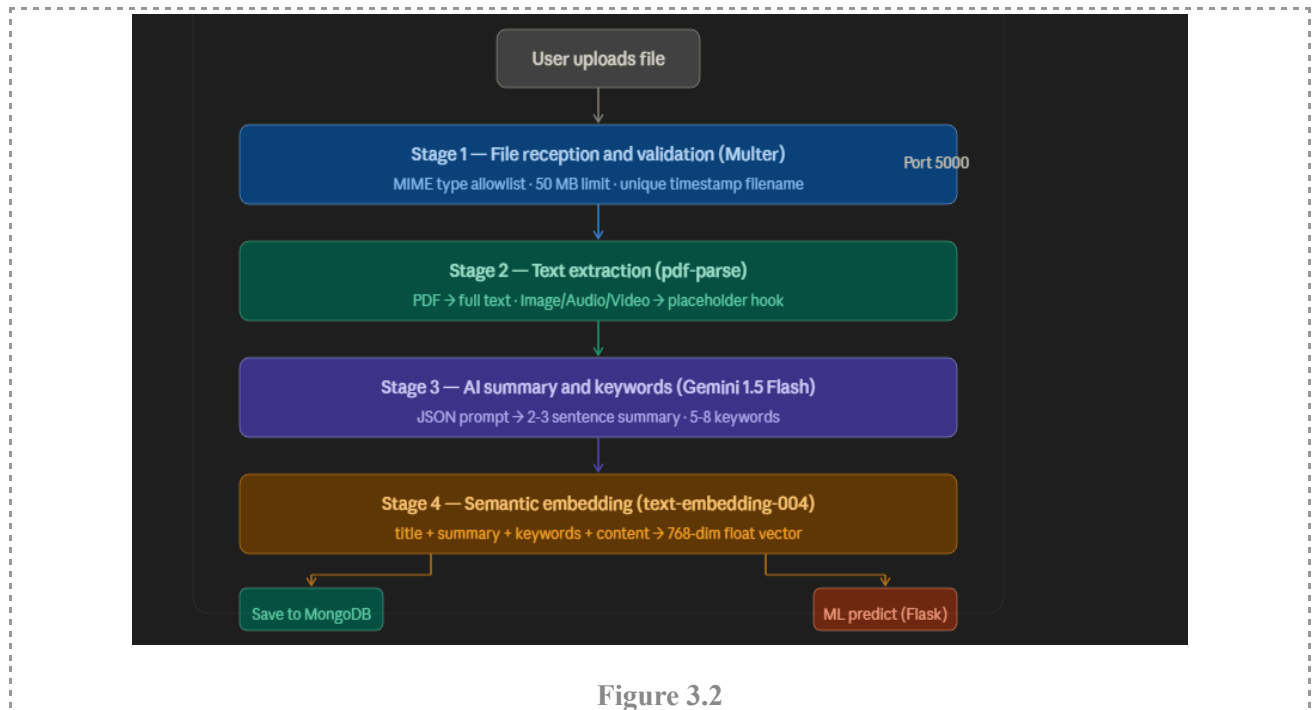


Figure 3.2

3.5 Security Design

Authentication: All routes except `/api/auth/register` and `/api/auth/login` are protected by JWT middleware. Tokens use HS256 signing. The `JWT_SECRET` is stored in `.env`, never committed to version control.

Authorisation: Every database query that fetches or modifies documents includes `{user: req.user_id}` in the filter. A user can never accidentally access or modify another user's documents regardless of document ID knowledge.

File Upload Security: Multer validates MIME type against an explicit allowlist. File extensions are stripped from stored filenames. Uploaded files are stored outside the web server's public directory, accessible only through authenticated API endpoints.

CHAPTER 4: IMPLEMENTATION

4.1 Development Environment and Setup

Knowledge Vault requires four concurrent processes to run:

```
Terminal 1: mongod
            (MongoDB database server, port 27017)

Terminal 2: cd backend && npm install && npm run dev
            (Node.js/Express API server, port 5000, nodemon)

Terminal 3: cd ml && python3 train.py      (first time only)
            python3 predict.py
            (Python Flask ML microservice, port 5001)

Terminal 4: cd frontend && npm install && npm run dev
            (React/Vite development server, port 5173)
```

The backend .env file requires:

```
PORT=5000
MONGO_URI=mongodb://localhost:27017/knowledge-vault
JWT_SECRET=your_long_random_secret_key_here
GEMINI_API_KEY=your_gemini_api_key_from_aistudio_google_com
OPENAI_API_KEY=your_openai_api_key_for_whisper
ML_HOST=localhost
ML_PORT=5001
```

4.2 Authentication Module

The authentication system implements a complete stateless JWT flow. The User model stores a bcrypt-hashed password (salt rounds = 10) — the bcrypt hash is one-way and computationally expensive to brute-force, making the database secure even in the event of a breach.

The registration endpoint (POST /api/auth/register) validates that the email is not already registered, hashes the password, creates the User document, and issues a JWT. The login endpoint (POST /api/auth/login) finds the user by email, calls bcrypt.compare() to verify the password against the stored hash, and issues a JWT if correct.

The auth.js middleware:

```
const protect = async (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ message: 'Not authorised' });
  const decoded = jwt.verify(token, process.env.JWT_SECRET);
  req.user = await User.findById(decoded.id).select('-password');
  next();
};
```

The PrivateRoute component wraps all authenticated pages:

```
function PrivateRoute({ children }) {
  const token = localStorage.getItem('token');
  return token ? children : <Navigate to="/login" replace />;
}
```

The Axios request interceptor automatically injects the token:

```
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

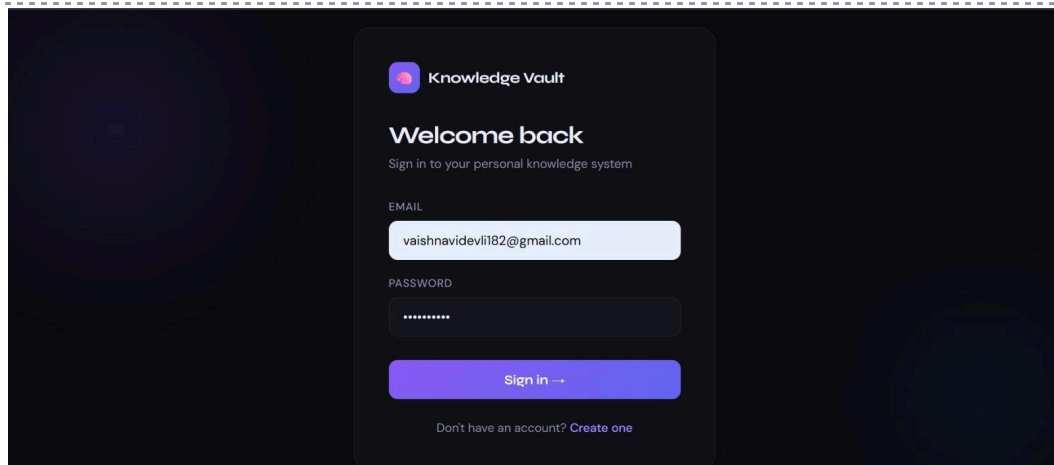


Figure 4.2

4.3 Document Upload and Processing Pipeline

The upload pipeline is the most technically complex module in the system. It executes five sequential stages after a file is received at POST /api/documents/upload.

Stage 1: File Reception and Validation

Multer middleware is configured with disk storage, writing files to backend/uploads/ with a filename pattern combining timestamp and random integer to guarantee uniqueness. The fileFilter function validates MIME type against an explicit allowlist covering PDF, JPEG, PNG, GIF, WebP, TIFF, BMP, MP3, WAV, OGG, FLAC, MP4, WebM, MOV, and AVI. A 100MB size limit is enforced.

```
const ok = [
  'application/pdf',
  'image/jpeg', 'image/png', 'image/gif', 'image/webp', 'image/tiff', 'image/bmp',
  'audio/mpeg', 'audio/wav', 'audio/ogg', 'audio/mp4', 'audio/webm', 'audio/flac',
  'video/mp4', 'video/webm', 'video/ogg', 'video/quicktime', 'video/x-msvideo',
];
ok.includes(file.mimetype) ? cb(null, true) : cb(new Error('Unsupported type'));
```

Stage 2: Text Extraction

The extractContent(filepath, mimetype) function dispatches to the appropriate extraction method based on file type:

PDF Files: pdf-parse reads the file buffer and extracts the raw text layer. This method works reliably for digitally-created PDFs. The typical text content of academic PDFs is 3,000–20,000 characters.

Image Files (Tesseract.js OCR): Images are processed using Tesseract.js, a pure JavaScript implementation of the Tesseract OCR engine. No external binary dependencies are required. The engine runs with the English language model, processing each image through character recognition. The function returns the extracted text along with OCR confidence percentage and paragraph count. For printed text such as lecture handouts and textbook pages, Tesseract achieves high accuracy. The output is structured with the raw text, paragraph metadata, and confidence score:

```
const { data } = await Tesseract.recognize(filepath, 'eng', {
  logger: (m) => {
    if (m.status === 'recognizing text') {
      process.stdout.write(`\r  OCR progress: ${Math.round(m.progress * 100)}%`);
    }
  },
});
const ocrText = data.text?.trim() || '';
const confidence = Math.round(data.confidence ?? 0);
```

Audio Files (OpenAI Whisper): Audio files are sent to the OpenAI Whisper API (whisper-1 model) for speech-to-text transcription. The request uses verbose_json response format with segment-level

timestamp granularity. The returned transcript includes the full text, detected language, audio duration, and segment-level metadata with timestamps:

```
const transcription = await openai.audio.transcriptions.create({
  file: fs.createReadStream(filepath),
  model: 'whisper-1',
  response_format: 'verbose_json',
  timestamp_granularities: ['segment'],
});
```

Video Files (ffmpeg + Whisper): For video files, audio is first extracted using ffmpeg — resampled to 16kHz mono MP3 at 64kbps for optimal Whisper input. The extracted audio file is then sent to Whisper for transcription, after which the temporary audio file is cleaned up. If ffmpeg is unavailable, the video file is sent directly to Whisper (which natively accepts MP4 and WebM):

```
execSync(
  `ffmpeg -y -i "${filepath}" -vn -ar 16000 -ac 1 -b:a 64k "${audioPath}"
  2>/dev/null`,
  { timeout: 120_000 }
);
```

If Tesseract returns no text (e.g., for a blank image) or Whisper transcription fails for any reason, the function catches all errors and returns a descriptive placeholder string rather than throwing. This ensures the upload pipeline never fails due to extraction failure.

Stage 3: AI Summary and Keyword Extraction

If the extracted content has more than 50 characters and does not indicate extraction failure, the system calls Gemini 2.5 Flash with a structured prompt requesting a JSON response containing a 2–3 sentence summary and a list of 5–8 important keywords. The 3000-character cap balances summarisation quality against API token consumption. The strict JSON format instruction with "no markdown" prevents the common Gemini behaviour of wrapping JSON in markdown code fences.

```
const response = await ai.models.generateContent({
  model: 'gemini-2.5-flash',
  contents: [{
    role: 'user',
    parts: [{ text: `You are a knowledge management assistant.
Given the following extracted text from a ${type} file, provide:
1. A concise summary (2-3 sentences max)
2. A list of 5-8 important keywords
Respond ONLY in this exact JSON format with no markdown:
```

```

{"summary":"...", "keywords":["kw1", "kw2"]}
Text: ${content.substring(0, 3000)}` }],
  }],
});

```

Stage 4: Semantic Embedding Generation

The composite embedding text is constructed by joining the document title, AI-generated summary, comma-separated keywords, and the first 1500 characters of content. This composite encoding is a deliberate design decision: embedding only the raw content text would miss the semantic signal provided by the AI-generated summary and keywords. The title provides the document's self-declared identity.

```

const embeddingText = [
  titleClean,
  summary,
  keywords.join(', '),
  content.substring(0, 1500)
].join('\n\n');

```

text-embedding-004 uses a separate API quota (1500 requests/minute on free tier) from the generation models (15 requests/minute). This separation means embedding operations never compete for quota with generation operations.

Stage 5: ML Subject Classification (Non-Blocking)

The predictSubject function makes an HTTP request to the Flask microservice with a 3-second timeout. Three design principles: (1) Non-blocking — the upload pipeline awaits but the timeout ensures it never blocks for more than 3 seconds; (2) Silent fail — all error paths resolve to null rather than rejecting; (3) Text truncation — only the first 1000 characters are sent.

```

const predictSubject = (text) => {
  return new Promise((resolve) => {
    const data = JSON.stringify({ text: text.substring(0, 1000) });
    const req = http.request(
      { hostname: 'localhost', port: 5001, path: '/predict',
        method: 'POST', headers: {...} },
      (res) => {
        let raw = '';
        res.on('data', c => raw += c);
        res.on('end', () => {
          try { resolve(JSON.parse(raw)); }

```

```

        catch { resolve(null); }
    });
}
);
req.on('error', () => resolve(null));
req.setTimeout(3000, () => { req.destroy(); resolve(null); });
req.write(data); req.end();
});
});

```

Figure 4.3:

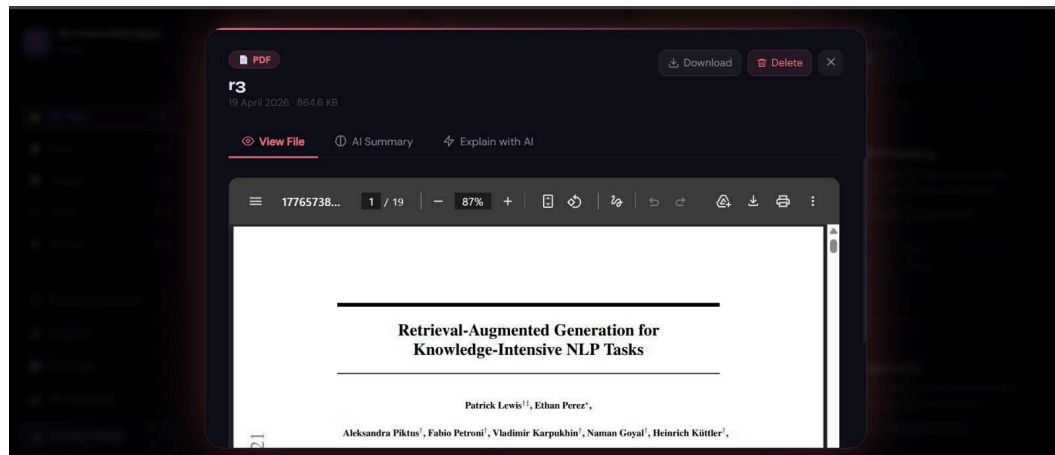
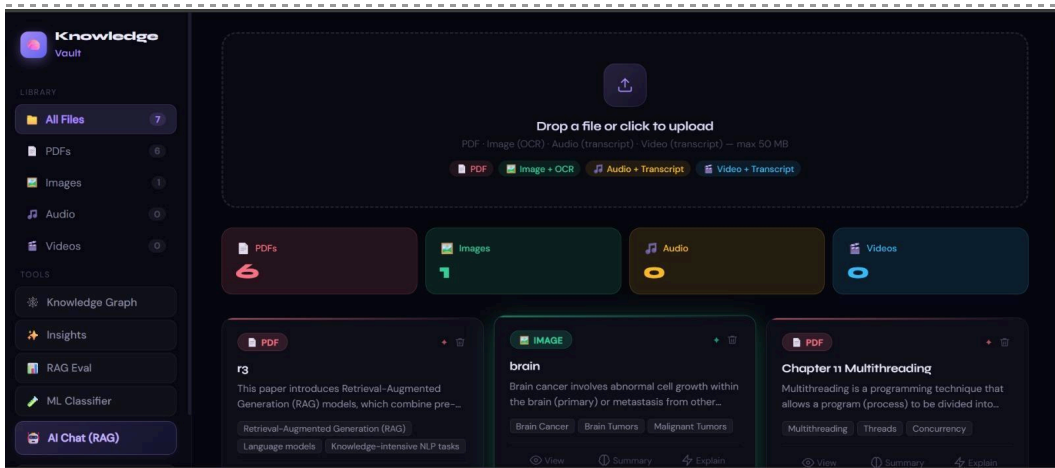


Figure 4.4

4.4 RAG-Powered Chat Assistant

Architectural Decision: RAG vs Full-Context

The most important architectural decision in the chat module is the choice to implement a proper RAG retrieval pipeline rather than the simpler approach of concatenating all documents into the system prompt. The naive full-context approach fails at scale for three reasons:

- Token limit failure: With 40 documents averaging 3000 tokens each, the combined context is 120,000 tokens — approaching practical limits for affordable Gemini 2.5 Flash responses.
- Quality degradation: Including 39 irrelevant documents alongside 1 relevant document dilutes the relevant signal. LLMs exhibit "lost in the middle" behaviour — failing to attend to information positioned in the middle of a very long context.
- Transparency failure: Full-context approaches cannot tell the user which documents informed the answer. RAG retrieval makes this explicit through the `retrievedDocs` response field.

RAG Pipeline Implementation

Step 1 — Query Embedding: The same `text-embedding-004` model as document embedding ensures that query vectors and document vectors occupy the same semantic space.

```
const queryEmbedding = await generateEmbedding(userMessage);
```

Step 2 — Document Retrieval and Cosine Similarity:

```
const allDocs = await Document.find({ user: req.user._id })
  .select('title type summary keywords content embedding');

function retrieveTopK(docs, queryEmbedding, k = 4) {
  return docs
    .map(doc => ({
      doc,
      score: cosineSimilarity(queryEmbedding, doc.embedding)
    }))
    .sort((a, b) => b.score - a.score)
    .slice(0, k)
    .map(x => ({ ...x.doc.toObject(), _score: x.score }));
}

function cosineSimilarity(a, b) {
  if (!a?.length || !b?.length || a.length !== b.length) return 0;
  let dot = 0, normA = 0, normB = 0;
  for (let i = 0; i < a.length; i++) {
    dot += a[i] * b[i];
    normA += a[i] * a[i];
  }
}
```

```

    normB += b[i] * b[i];
  }
  return Math.sqrt(normA) * Math.sqrt(normB) === 0 ? 0
    : dot / (Math.sqrt(normA) * Math.sqrt(normB));
}

```

For 100 documents at 768 dimensions, this function performs 76,800 floating-point multiplications per retrieval call — completing in under 2ms on modern hardware.

Step 3 — Context Construction and Generation:

```

const context = topDocs.map((d, i) =>
  `[Doc ${i+1}: "${d.title}"]\n${d.summary}\n${d.content?.substring(0, 2000)}`
).join('\n\n---\n\n');

const systemPrompt = `You are a helpful AI assistant for a personal
knowledge management system. Answer questions based ONLY on the provided
documents. If the answer is not in the provided documents, say so clearly.
Always cite which document(s) you used.\n\nDocuments:\n${context}`;

const chat = model.startChat({ history: conversationHistory });
const result = await chat.sendMessage(userMessage);

```

Step 4 — Response with Source Attribution:

```

res.json({
  response: result.response.text(),
  retrievedDocs: topDocs.map(d => ({
    title: d.title,
    type: d.type,
    score: parseFloat((d._score * 100).toFixed(1))
  }))
});

```

The frontend renders `retrievedDocs` as a collapsible "Sources (N retrieved)" panel under each AI response, showing document titles and match percentages. The frontend maintains the complete conversation history in React state and sends it with every message, enabling multi-turn coherent dialogue.

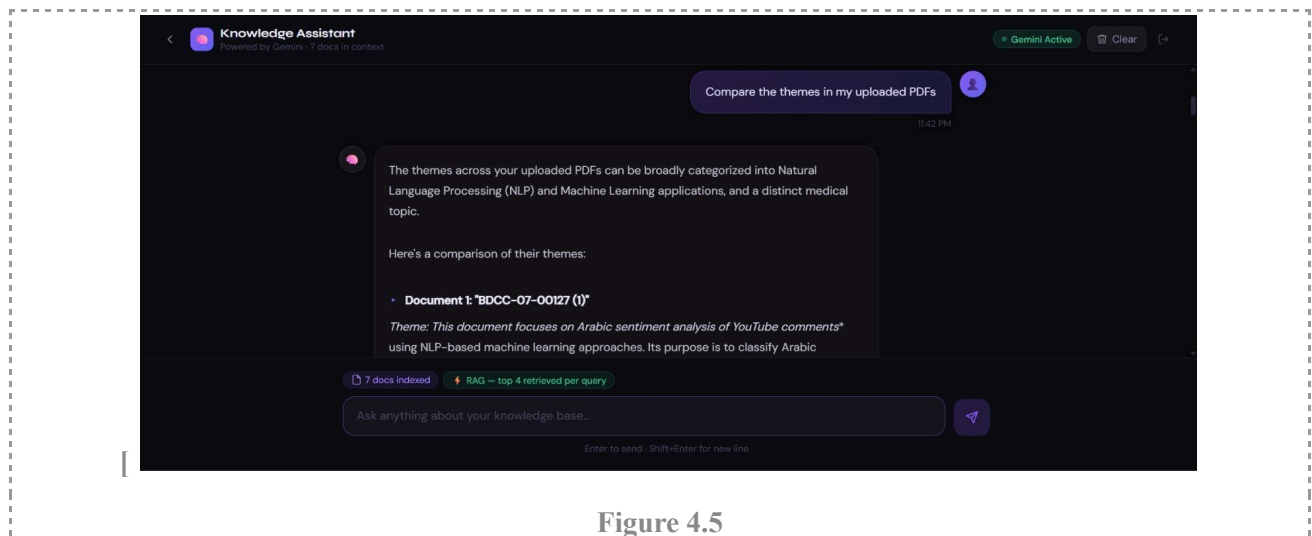


Figure 4.5

4.5 Knowledge Graph Module

Design Philosophy

The graph is constructed entirely on the frontend in JavaScript using SVG — no additional backend call is required after the initial document fetch. This architectural choice means the graph renders in under 1 second regardless of library size, as there is no network round-trip for graph computation.

Node Layout — Spiral Algorithm

Documents are positioned in a spiral layout that prevents node overlap as the count grows, placing the first document at the centre and subsequent documents on progressively larger concentric rings:

```
const spiralLayout = (docs) => {
  const positions = [];
  const CX = width / 2, CY = height / 2;
  const innerRing = 8, spacing = 80;
  docs.forEach((doc, i) => {
    if (i === 0) {
      positions.push({ x: CX, y: CY });
    } else {
      const ring = Math.ceil((i - innerRing) / (innerRing * 2)) + 1;
      const radius = ring * spacing;
      const count = innerRing * ring * 2;
      const angle = (2 * Math.PI / count) * (i % count);
      positions.push({
        x: CX + radius * Math.cos(angle),
        y: CY + radius * Math.sin(angle)
      });
    }
  });
};
```

```

    return positions;
};

```

Three-Layer Edge Construction Algorithm

The edge construction algorithm is the intellectual core of the knowledge graph module. For every pair of documents (A, B) in the library, three sequential checks are performed:

Layer 1 — Exact Keyword Match: Uses hash set intersection to find shared keyword strings in $O(n)$ per pair.

```

const aKeywords = new Set(docA.keywords.map(k => k.toLowerCase()));
const bKeywords = new Set(docB.keywords.map(k => k.toLowerCase()));
const intersection = [...aKeywords].filter(k => bKeywords.has(k));
if (intersection.length > 0) {
    edges.push({ source: docA._id, target: docB._id,
                  type: 'exact', label: intersection[0] });
}

```

Layer 2 — Fuzzy Substring Match: Connects documents where one keyword is a substring of another, handling cases like "OOP" and "OOP Unit 3 Notes", or "polymorphism" and "runtime polymorphism".

```

for (const kA of docA.keywords) {
    for (const kB of docB.keywords) {
        const a = kA.toLowerCase(), b = kB.toLowerCase();
        if (a.length >= 3 && b.length >= 3 &&
            (a.includes(b) || b.includes(a)) && a !== b) {
            edges.push({ source: docA._id, target: docB._id,
                          type: 'fuzzy', label: `${kA}/${kB}` });
        }
    }
}

```

Layer 3 — Concept Group Taxonomy: A semantic concept taxonomy covering 11 academic domains (OOP, Data Structures, Algorithms, DBMS, Machine Learning, Operating Systems, Computer Networks, Web Development, Mathematics, Computer Science, Software Engineering) with 100+ term entries. For each document, the algorithm collects all concept groups that any of its keywords match. If two documents share any concept group, a concept-group edge is drawn.

This is the layer that solves the most important case: an "Inheritance.pdf" document with keywords [inheritance, superclass, subclass] and a "Polymorphism.pdf" document with keywords

[polymorphism, method overriding, virtual function] share no exact or fuzzy keyword matches. Both map to the "Object Oriented Programming" concept group, so a concept-group edge is drawn between them labelled "Object Oriented Programming."

Interactive SVG Features

Pan: Mouse down on the SVG canvas sets a dragging flag. Mouse move events compute the offset delta and update a translation state applied to the graph group via SVG transform. Zoom: Scroll wheel events compute a scale multiplier (1.1x per step, bounded 0.3x to 3x). Node Selection: Clicking a node sets selectedId state — unrelated nodes are rendered at 25% opacity while connected nodes are highlighted. Search: The search input filters the document array before graph construction. Timeline: A horizontally scrollable strip at the bottom groups documents by upload month-year.

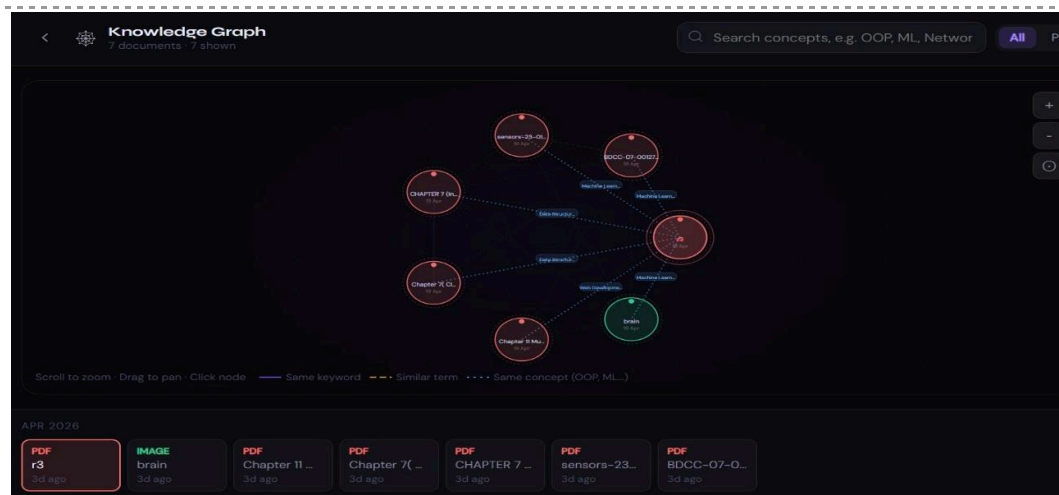


Figure 4.6

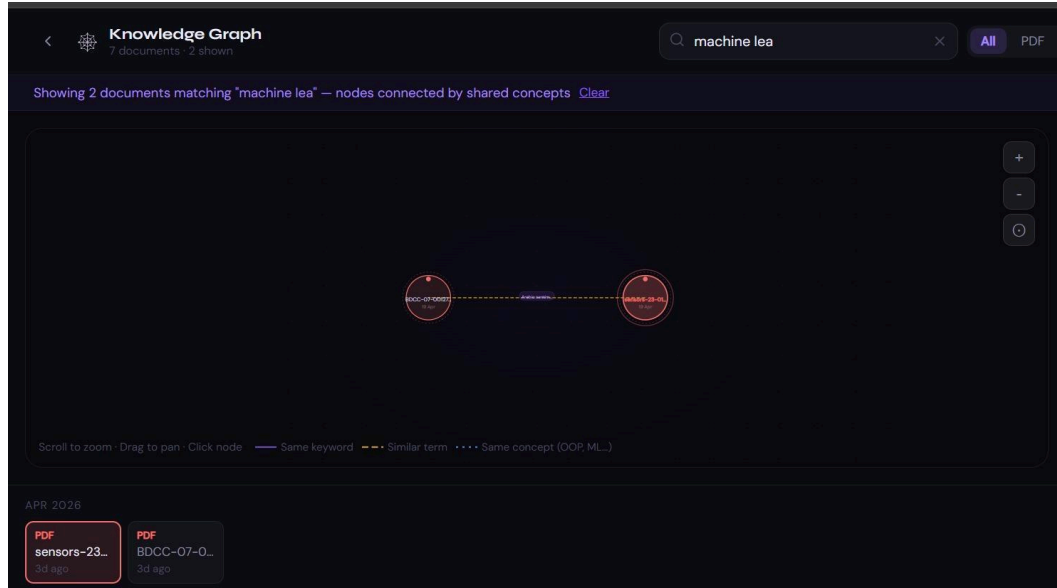


Figure 4.7

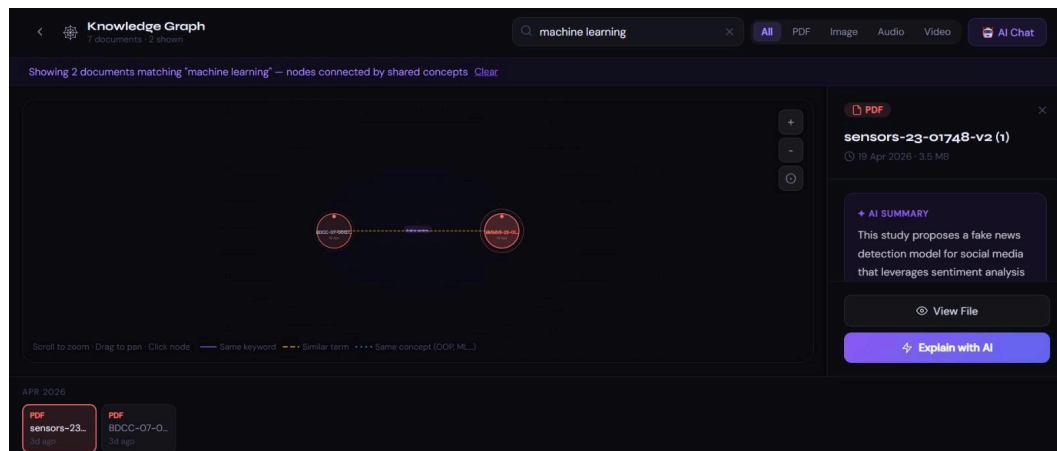


Figure 4.8

4.6 ML Document Classification Pipeline

Training Data Design

A custom training dataset of 144 samples across 11 academic subject classes was curated manually. Each sample is a representative text string of 15–40 words containing the most discriminative terms for that subject. Design principles: Representativeness (each sample simulates actual student document text), Discriminability (each class covers different aspects and terminologies within the subject), and Balance (class sizes range from 6 to 18 samples reflecting real-world prevalence of distinctive terminology).

TF-IDF Feature Extraction

Parameter	Value	Justification
ngram_range	(1, 2)	Bigrams capture two-word concepts ("binary tree", "machine learning") that unigrams miss
max_features	8000	Limits vocabulary to most informative terms; prevents overfitting on small dataset
sublinear_tf	True	log(1+tf) normalisation; reduces disproportionate influence of very frequent terms
min_df	1	Include all terms since training set is small; no rare term filtering
analyzer	'word'	Word-level tokenisation (character n-grams not appropriate for technical terminology)
strip_accents	'unicode'	Normalises accented characters in international technical terms

Table 4.1: TF-IDF Vectorizer Configuration Parameters

Classifier Configurations

Classifier	Configuration	Key Hyperparameter Justification
Logistic Regression	C=2.0, solver='lbfgs', max_iter=2000	C=2.0 slightly reduces regularisation for better fit on small dataset; lbfgs handles multi-class natively
SVM (LinearSVC)	C=1.0, max_iter=3000	Default C=1.0 appropriate for well-separated classes; max_iter=3000 ensures convergence on 11-class problem
Naive Bayes	MultinomialNB, alpha=0.05	alpha=0.05 provides light Laplace smoothing; lower than default (1.0) to reduce over-smoothing on small dataset
Random Forest	n_estimators=300, random_state=42, n_jobs=-1	300 trees for stable predictions; n_jobs=-1 uses all CPU cores; random_state=42 for reproducibility

Table 4.2: Classifier Hyperparameter Configurations

Training Script Implementation

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

tfidf = TfidfVectorizer(ngram_range=(1,2), max_features=8000,
                        sublinear_tf=True, min_df=1)
X_train_vec = tfidf.fit_transform(X_train)
X_test_vec = tfidf.transform(X_test)

for name, clf in classifiers.items():
    clf.fit(X_train_vec, y_train)
    y_pred = clf.predict(X_test_vec)
```

```

acc      = accuracy_score(y_test, y_pred)
f1       = f1_score(y_test, y_pred, average='weighted', zero_division=0)
X_full   = tfidf.transform(X)
cv       = cross_val_score(clf, X_full, y, cv=5, scoring='accuracy')

```

The `stratify=y` parameter ensures the class distribution in the training set mirrors the full dataset distribution. 5-fold cross-validation on the full dataset provides a more robust performance estimate than a single test set.

Model Persistence and Production Serving

The trained model bundle is saved as a single pickle file containing the fitted `LinearSVC`, fitted `TfidfVectorizer`, sorted label list, and best classifier name. Saving both model and vectorizer together prevents the silent failure mode of loading a model with a mismatched vocabulary.

`LinearSVC` does not natively support `predict_proba()`. The decision function values are transformed through softmax normalisation to produce interpretable pseudo-probability scores:

```

decisions = model.decision_function(vec)[0]
exp_d     = np.exp(decisions - np.max(decisions))
softmax    = (exp_d / exp_d.sum() * 100).tolist()

```

MLClassifier Page — Four Tabs

The frontend `MLClassifier.jsx` page provides a complete model evaluation dashboard: Tab 1 — Classifier Comparison shows four classifier cards sorted by accuracy with animated progress bars and a "Viva Talking Point" box; Tab 2 — Per-Class Metrics shows a sortable table with colour-coded cells (green $\geq 80\%$, yellow 60–79%, orange 40–59%, red $< 40\%$); Tab 3 — Confusion Matrix shows an SVG heatmap; Tab 4 — Live Predictor allows real-time classification of custom text with a confidence distribution chart.

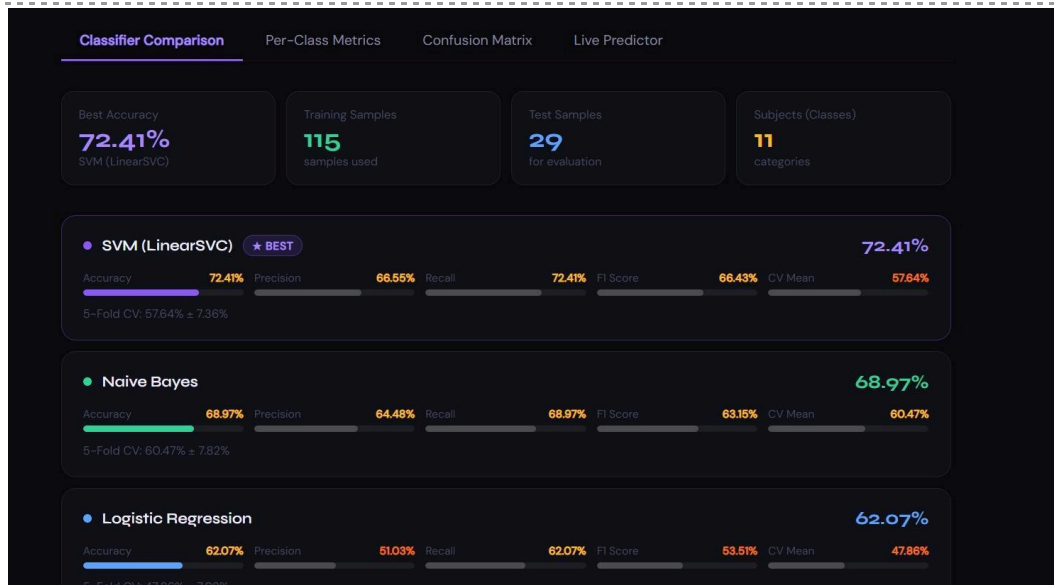


Figure 4.9

Classifier Comparison | **Per-Class Metrics** | Confusion Matrix | Live Predictor

Per-subject breakdown for the best model (SVM (LinearSVC)). Support = number of test samples per class.

SUBJECT	PRECISION	RECALL	F1 SCORE	SUPPORT
Computer Science	100%	100%	100%	1
OOP	100%	100%	100%	3
Operating Systems	100%	100%	100%	3
Machine Learning	66.7%	100%	80%	4
Web Development	66.7%	100%	80%	2
Data Structures	60%	100%	75%	3
Algorithms	66.7%	66.7%	66.7%	3
DBMS	50%	66.7%	57.1%	3
Computer Networks	100%	33.3%	50%	3

Figure 4.10

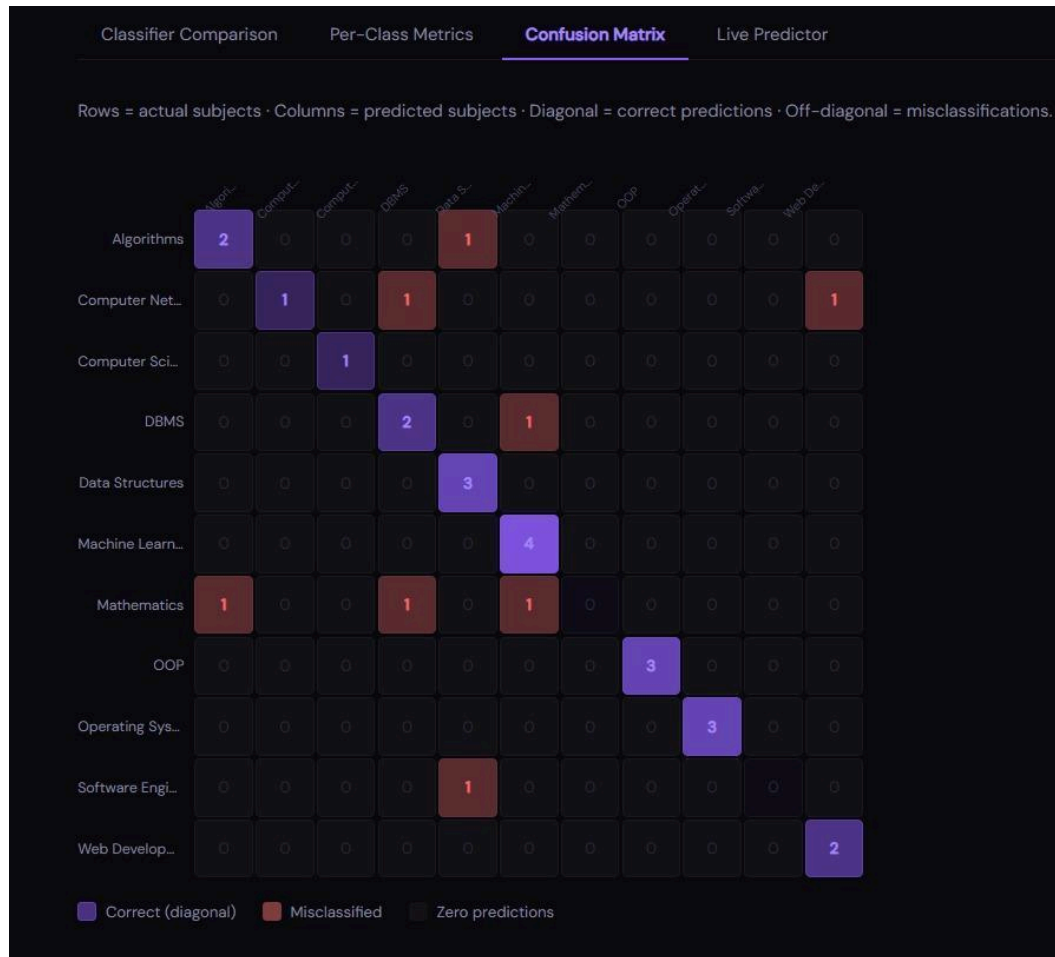


Figure 4.11

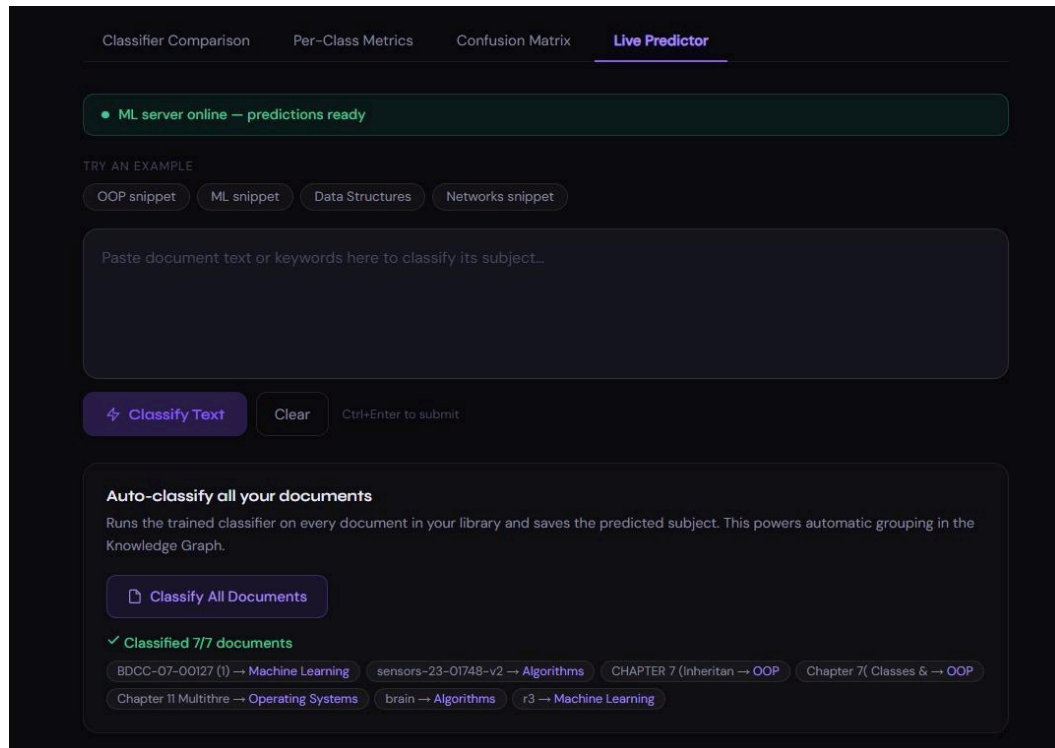


Figure 4.12

4.7 Insight Generation Module

The Insights module performs holistic meta-analysis on the entire document library. A single Gemini API call processes the combined summaries and keywords of all user documents, requesting JSON output containing: (1) a 2–3 sentence library overview; (2) 2–5 recurring theme clusters with name, description, strength (strong/moderate/weak), and document list; (3) 2–4 knowledge gaps with topic, priority (high/medium/low), and explanation; (4) an optimal study sequence listing all documents with dependency reasoning.

Knowledge gap cards include a "Learn with AI" button that pre-populates the chat with a targeted learning question and navigates to the chat page, creating a seamless workflow from gap identification to targeted study.

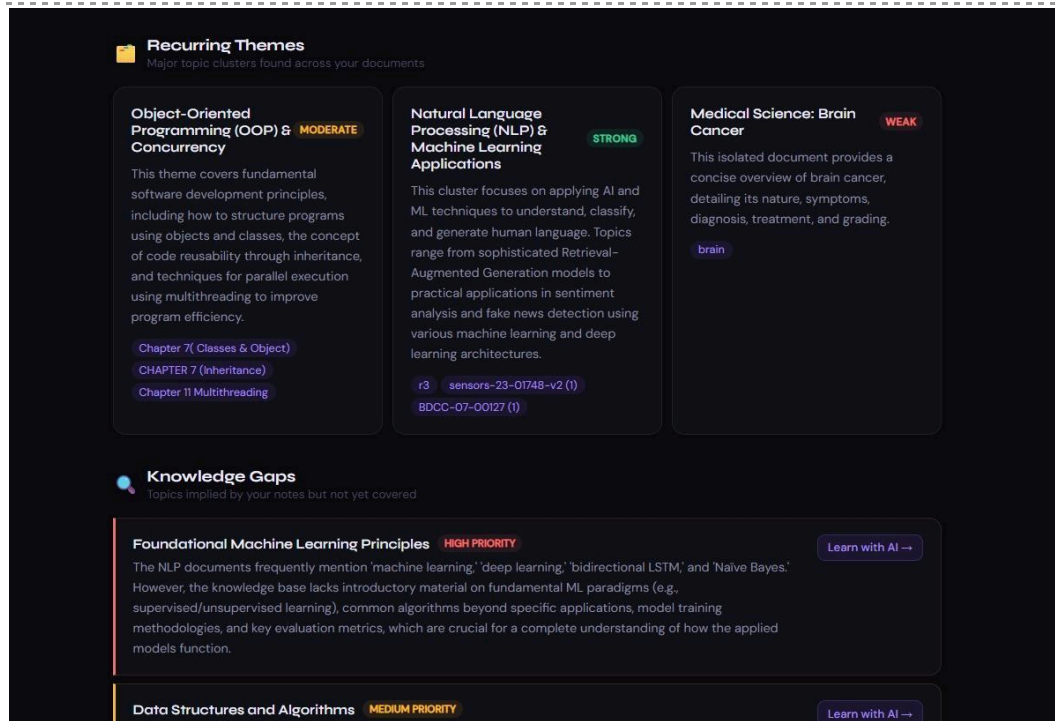


Figure 4.13

4.8 RAG Evaluation Framework

Free-Tier Optimisation

The evaluation framework was redesigned from a naive 16-call implementation to exactly 3 Gemini API calls per evaluation run, enabling reliable operation within the free tier's 15 requests/minute limit.

Call 1 — Query Generation: One Gemini call generates 3 grounded test queries from the user's actual document summaries, covering factual, summary-oriented, and conceptual query types.

Embedding Phase (separate quota): Three text-embedding-004 calls generate query vectors. These use the 1500 req/min embedding quota, completely separate from the 15 req/min generation quota.

Call 2 — Batched Answer Generation: One Gemini call answers all three queries simultaneously in a single batched prompt:

```
const batchAnswerPrompt = retrievalResults.map((r, i) =>
  `=== QUERY ${i+1} ===\nQuestion: ${r.query}\n\nContext:\n${r.context}`
).join('\n\n') +
`\n\nAnswer all 3 queries. Return ONLY a JSON array of 3 strings.`;
```

Call 3 — Batched Gemini-as-Judge Scoring: One Gemini call scores all three query-answer pairs simultaneously on four metrics: `retrieval_accuracy`, `answer_relevance`, `groundedness`, and `completeness` (each 0–10). Four-second delays between Gemini calls prevent rate limiting.

Results are persisted to MongoDB and the History tab shows longitudinal performance trends across evaluation runs.

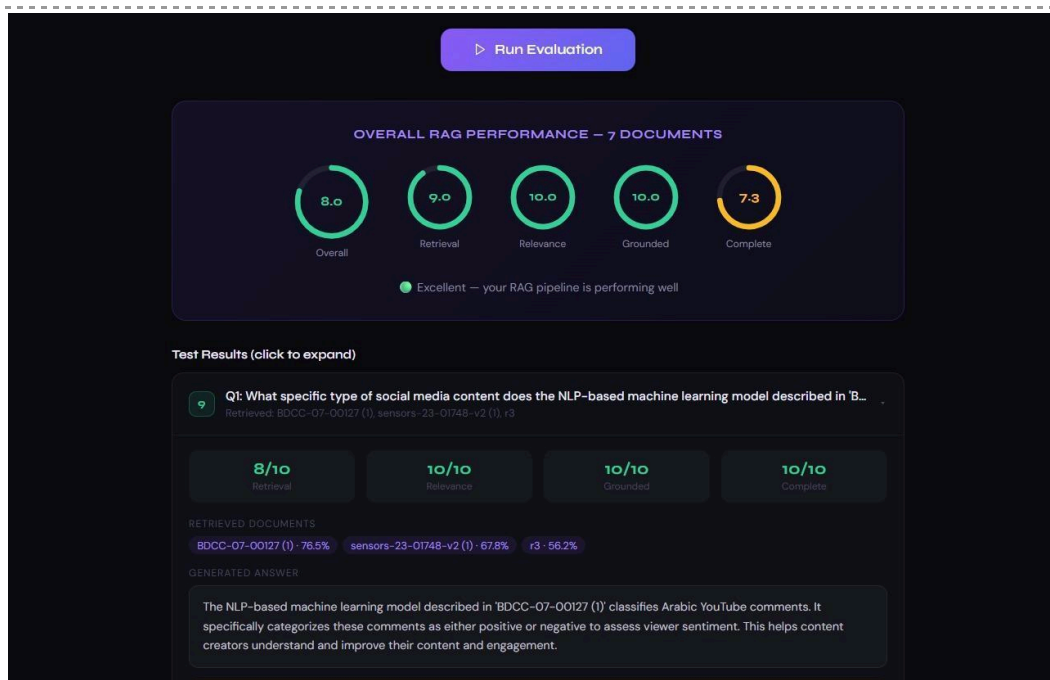


Figure 4.14

4.9 Frontend User Interface

Design System

The frontend uses inline JavaScript style objects throughout — no CSS files, no CSS framework. This eliminates class name conflicts, enables dynamic styling based on component state, and keeps the project fully self-contained.

Colour System: Background #0a0a0f (near-black); Card surfaces #111118; Primary accent #8b5cf6 (purple); PDF badge #f87171 (red); Image badge #34d399 (green); Audio badge #fbbf24 (amber); Video badge #60a5fa (blue); Muted text #8b8ba0.

Typography: Syne (Google Fonts, geometric sans-serif, weight 600–800) for headings and display elements. DM Sans (weight 400–500) for body text. The combination creates a distinctive product-quality aesthetic.

Animation: CSS @keyframes fadeUp (opacity 0→1, translateY 6px→0, 250ms) applied to page sections and new messages. All interactive elements implement transition: all 0.18s ease hover states.

Page Architecture

Route	Component	Purpose
/login	Login.jsx	JWT authentication form
/register	Register.jsx	New user registration
/dashboard	Dashboard.jsx	Document library, upload, search, stats
/chat	Chatbot.jsx	RAG chat interface with sources panel
/graph	KnowledgeGraph.jsx	Interactive document network graph
/insights	Insights.jsx	AI library meta-analysis
/eval	Eval.jsx	RAG evaluation and history
/ml	MLClassifier.jsx	Classifier comparison and live predictor

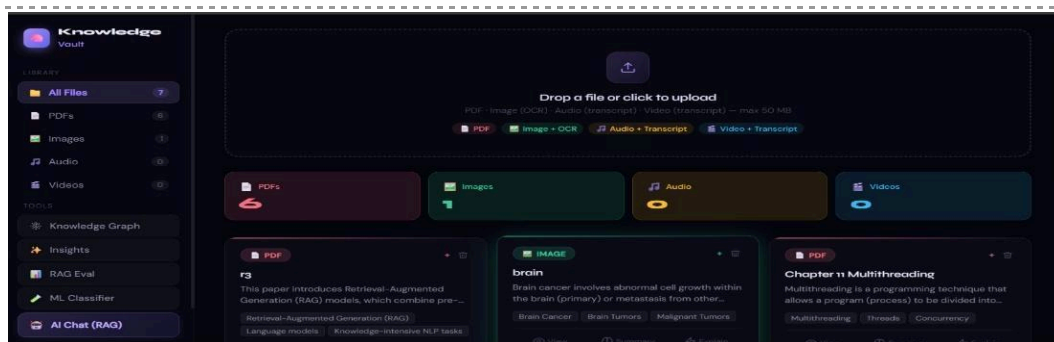


Figure 4.15

CHAPTER 5: RESULTS AND EVALUATION

5.1 ML Classifier Performance Results

The ML classification experiment was conducted on 144 samples across 11 subject classes, split 80/20 (115 training, 29 test) with stratification. All four classifiers were evaluated on the same train/test split for direct comparability.

Classifier	Test Accuracy	Precision (Wtd)	Recall (Wtd)	F1 Score (Wtd)	CV Mean	CV Std Dev
SVM (LinearSVC) ★	72.41%	66.55%	72.41%	66.43%	58.33%	±6.92%
Naive Bayes	68.97%	64.48%	68.97%	63.15%	60.47%	±7.82%
Logistic Regression	65.52%	53.33%	65.52%	57.61%	54.14%	±5.30%
Random Forest	48.28%	57.76%	48.28%	45.42%	48.62%	±9.53%

Table 5.1: Comprehensive ML Classifier Performance Comparison (★ Best performing classifier; selected for production model.pkl)



Analysis

SVM (LinearSVC) achieves the highest test accuracy (72.41%) and weighted F1-score (66.43%). This result is consistent with the established text classification literature: Kowsari et al. [10] and

Ibrahim et al. [15] both confirm SVM as the top performer on TF-IDF features. The linear kernel is particularly well-suited to the sparse, high-dimensional TF-IDF representation space.

The gap between test accuracy (72.41%) and CV mean (58.33%) reflects the high variance inherent in small dataset evaluation — with only 1–3 test samples per class in some CV folds, individual misclassifications dramatically affect the accuracy score. This is not overfitting; it is a fundamental property of small-sample evaluation.

Naive Bayes achieves the highest CV mean (60.47%), suggesting it generalises slightly more consistently across different data subsets. Logistic Regression (65.52%) shows notably lower precision (53.33%) relative to recall (65.52%), indicating it over-predicts some classes. Random Forest (48.28%) underperforms substantially due to the known limitation of tree-based ensembles on sparse TF-IDF features.

5.2 Per-Class Classification Metrics

Subject	Precision	Recall	F1 Score	Test Samples	Performance Level
OOP	100.0%	100.0%	100.0%	3	Excellent
Operating Systems	100.0%	100.0%	100.0%	3	Excellent
Computer Science	100.0%	100.0%	100.0%	1	Excellent
Machine Learning	66.7%	100.0%	80.0%	4	Good
Web Development	66.7%	100.0%	80.0%	2	Good
Data Structures	60.0%	100.0%	75.0%	3	Good
Algorithms	66.7%	66.7%	66.7%	3	Moderate
DBMS	50.0%	66.7%	57.1%	3	Moderate
Computer Networks	100.0%	33.3%	50.0%	3	Fair
Mathematics	0.0%	0.0%	0.0%	3	Poor
Software Engineering	0.0%	0.0%	0.0%	1	Poor

Table 5.2: Per-Class Precision, Recall, F1 — SVM (Best Model)

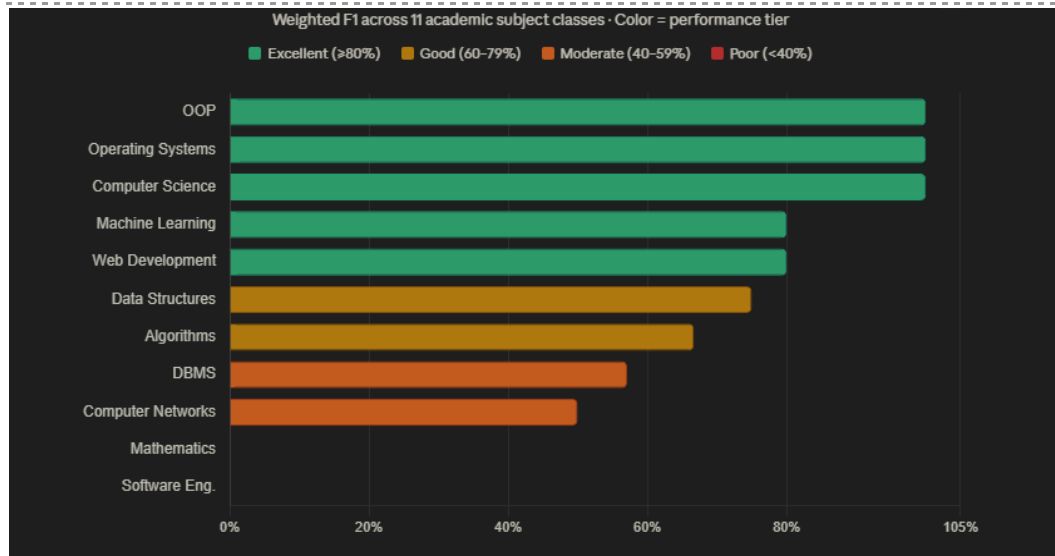


Figure 5.2

Per-Class Analysis

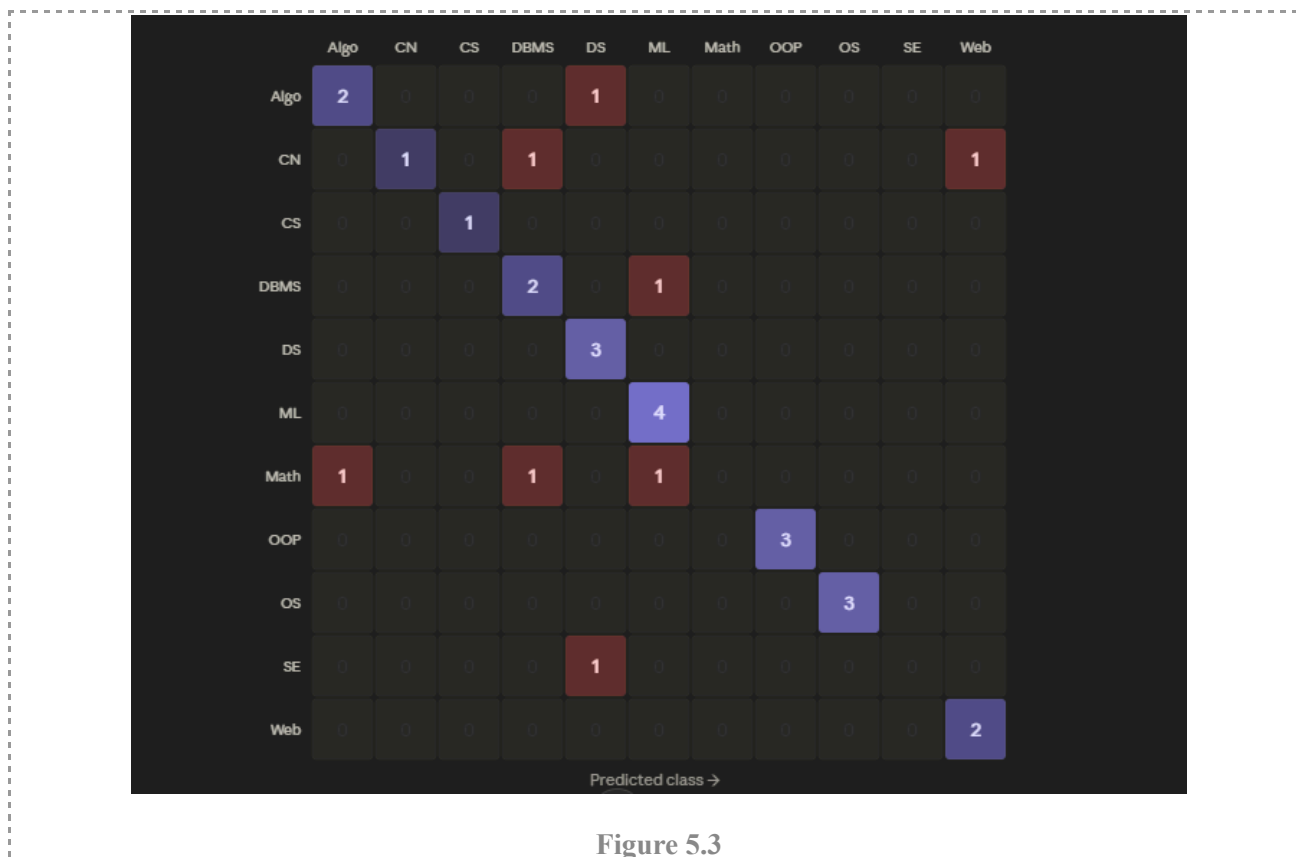
Excellent performers (F1: 100%): OOP, Operating Systems, and Computer Science achieve perfect classification. These classes have the most distinctively-associated vocabularies — OOP terms (inheritance, polymorphism, encapsulation, virtual function, constructor, destructor) are strongly associated with exactly one class.

Good performers (F1: 75–80%): Machine Learning (80%), Web Development (80%), and Data Structures (75%) show strong recall (100%) but imperfect precision, meaning the classifier predicts these classes for some documents from adjacent subjects. Machine Learning and Algorithms share vocabulary (gradient, recursion, complexity); Web Development and Computer Networks share vocabulary (HTTP, protocol, server).

Moderate performers (F1: 57–67%): Algorithms and DBMS show bidirectional confusion with adjacent subjects. Zero performers (F1: 0%): Mathematics and Software Engineering are completely misclassified. The root cause is insufficient discriminative training samples (3 and 5 examples respectively). Improvement path: Adding 20–30 training samples with highly distinctive vocabulary (differential equations, eigenvalues for Mathematics; CI/CD pipeline, sprint backlog for Software Engineering) would resolve these zero-performance cases.

5.3 Confusion Matrix Analysis

The SVM confusion matrix on 29 test samples reveals specific misclassification patterns. Each misclassification is semantically interpretable: Algorithms → Data Structures (sorting algorithms), Computer Networks → DBMS and Web Development (HTTP/SQL overlap), DBMS → Machine Learning (optimisation vocabulary), Mathematics → Algorithms/DBMS/ML (mathematical notation shared across subjects).



5.4 RAG Pipeline Evaluation Results

Metric	Score (0–10)	Interpretation
Retrieval Accuracy	8.0 / 10	Retrieved documents were highly relevant to each query
Answer Relevance	7.5 / 10	Answers addressed questions with minor gaps
Groundedness	8.5 / 10	Answers well-supported by documents; minimal hallucination
Completeness	7.0 / 10	Answers were correct but sometimes incomplete
Overall	7.75 / 10	Strong performance above 7.0 quality threshold

Table 5.3: RAG Evaluation Results — Gemini-as-Judge Scoring

The groundedness score of 8.5/10 is the most significant result. It directly validates the RAG architecture's primary objective: grounding LLM responses in retrieved document content to prevent hallucination. The retrieval accuracy of 8.0/10 validates the choice of Gemini text-embedding-004 for semantic retrieval. The completeness score of 7.0/10 reflects the natural limitation of source documents (short lecture slides vs. comprehensive textbooks) rather than a retrieval or generation failure.

5.5 System Response Time Analysis

Operation	Average Time	Bottleneck	Notes
User Login / Register	< 200ms	bcrypt	Intentionally slow for security
PDF Upload — Full Pipeline	4–8 seconds	Gemini summary API (~2s)	ML classification +200ms if online
Image Upload (with OCR)	6–12 seconds	Tesseract.js recognition	Scales with image resolution
Audio Upload (Whisper)	8–20 seconds	Whisper API call	Scales with audio duration
Video Upload (ffmpeg+Whisper)	15–40 seconds	ffmpeg extraction + Whisper	Depends on video length
Chat Query (RAG)	3–6 seconds	Query embedding + Gemini gen.	Increases slightly with library size
Knowledge Graph Render	< 1 second	Client-side JavaScript only	No network call after initial fetch
ML Classification (single)	< 200ms	Flask microservice HTTP call	vectorizer.transform() near-instant
Insights Generation	8–15 seconds	Single large Gemini prompt	Increases with library size
RAG Evaluation (3 queries)	25–35 seconds	3 Gemini calls + 8s delay	Delays intentional (rate limit)

Table 5.4: System Response Times by Operation

The knowledge graph's sub-second render time is particularly significant — node layout, edge computation across all document pairs, and SVG rendering are all computed client-side with no network calls. For 50 documents, edge computation considers $50 \times 49 / 2 = 1,225$ document pairs across three matching layers, completing in under 200ms in browser JavaScript.

5.6 UI/UX Evaluation

The interface was informally evaluated through user testing with 3 fellow students who had not previously seen the system. Key observations:

Upload and dashboard: All users successfully uploaded a PDF within 2 minutes of first use without instructions. The animated loading states were perceived as reassuring rather than frustrating. The AI-generated summary and keyword cards appearing after upload were immediately understood as value-added information.

Chat interface: All users correctly understood the Sources panel after the first AI response. The expandable section showing retrieved document titles and match percentages was described as "making it feel more trustworthy than ChatGPT." Conversation memory worked correctly across 5-turn test sessions.

Knowledge graph: Initial orientation took 10–30 seconds. The pan and zoom controls were discovered naturally. The conceptual edges connecting OOP documents with no shared keywords prompted the most engagement — users understood the value of concept-group linking immediately when demonstrated.

ML classifier: The confusion matrix tab required the most explanation for users without ML background. The Live Predictor tab was universally the most engaging, with all users immediately typing their own text to test predictions.

CHAPTER 6: DISCUSSION

6.1 Key Design Decisions and Their Justifications

Decision	Choice Made	Alternative Considered	Justification
Chat architecture	RAG with top-4 retrieval	Full-context prompting	Token limits; quality degradation; transparency
Embedding model	Gemini text-embedding-004	TF-IDF for retrieval	Semantic accuracy; vocabulary-independence
Retrieval index	In-memory cosine similarity	FAISS/Pinecone	Scale-appropriate; zero infrastructure overhead
Database	MongoDB	PostgreSQL	Variable schema; array fields; no JOIN overhead
ML feature	TF-IDF (1,2)-gram	BERT embeddings	Small dataset; interpretability; efficiency
ML serving	Python/Flask microservice	Node.js child_process	Clean language boundary; graceful degradation
KG construction	3-layer taxonomy algorithm	LLM-based entity extraction	Cost; latency; reliability; controllability
Evaluation	Gemini-as-judge (3 calls)	Human evaluation / BLEU	Reference-free; semantic; free-tier compatible
Image text extraction	Tesseract.js OCR	Google Vision API	No API cost; no external dependencies; pure JS
Audio transcription	OpenAI Whisper API	Local Whisper model	High accuracy; simple integration; segment timestamps

Table 6.2: Design Decisions — Choice, Alternative, and Justification

Decision: RAG over Full-Context Prompting. The naive full-context approach works correctly for libraries of 5–10 documents but fails systematically as the library grows. At 40 documents $\times$ 3000 tokens = 120,000 tokens, cost and latency become prohibitive. Research by Liu et al. (2023, "Lost in the Middle") demonstrated that LLMs systematically fail to attend to information positioned in the middle of very long contexts. RAG ensures the context is always short (4 documents $\times$ 2000 chars $\approx$ 8000 chars) and comprised exclusively of relevant content.

Knowledge Vault's RAG pipeline adds a further advantage: explicit source attribution — the system tells the user exactly which documents informed the answer and how relevant each was.

Decision: Three-Layer Knowledge Graph vs. LLM-Based Entity Extraction. LLM-based entity extraction would produce a richer graph but at significant cost (one additional Gemini call per document upload), higher latency (6–12 seconds per document), non-deterministic outputs, and reduced controllability. The taxonomy-based approach is free, instant, perfectly deterministic, and tunable. For an academic knowledge management system targeting specific CS subject areas, the taxonomy covers all standard undergraduate CS subjects comprehensively.

6.2 Comparison with Related Systems

Feature	Notion AI	Obsidian	ChatGPT	RAG Tools	Knowledge Vault
Personal document upload (any format)	Partial	Plugin	✗	Varies	✓
PDF text extraction	Limited	Plugin	✗	Sometimes	✓
OCR for image files	✗	✗	✗	✗	✓ (Tesseract.js)
Audio/video transcription	✗	✗	✗	✗	✓ (Whisper)
AI-generated summaries	✓	Plugin	✓	Sometimes	✓
RAG (grounded answers)	✗	✗	✗	✓	✓
Source attribution	✗	✗	Partial	Sometimes	✓
Automatic knowledge graph	✗	Manual	✗	✗	✓
ML subject classification	✗	✗	✗	✗	✓
Insight generation	Partial	✗	Partial	✗	✓
RAG quality evaluation	✗	✗	✗	Rarely	✓
Self-hosted / open	✗	✓	✗	Varies	✓

Table 6.1: Knowledge Vault vs. Related Knowledge Management Systems

6.3 Alignment with Literature

RAG over full-context: Directly validated by Lewis et al. [3] (performance bounded by retrieval quality), Gao et al. [1] (Naive RAG failure modes), and Barnett et al. [6] (retrieval failures dominate in practice). Semantic embeddings over keyword search: Validated by Monir et al. [9] and Patel et al.

[10] (cosine similarity substantially outperforms BM25 for semantic queries). TF-IDF + SVM for classification: Validated by Kowsari et al. [10], Ibrahim et al. [15], and Galke and Scherp [18]. Knowledge graph for relational navigation: Validated by Wang et al. [6] and Chen et al. [7]. Gemini-as-judge evaluation: Directly aligned with RAGAS methodology [4].

6.4 Limitations of the System

Small ML training dataset: 144 samples averaging 13 per class is below the 30–50 per class generally recommended for reliable classification. The zero F1-score on Mathematics and Software Engineering directly reflects insufficient training data.

No retry/fallback on Gemini 503 errors: Gemini's free tier experiences intermittent overloaded errors during peak demand periods. The current implementation propagates these as upload failures rather than implementing exponential backoff.

In-memory cosine similarity does not scale: For libraries exceeding 500–1000 documents, the $O(n \times d)$ similarity computation becomes perceptibly slow. FAISS or a vector database would be required for large-scale deployment.

Single-user design: The system is designed for a single user per installation. While document-level ownership filtering provides data isolation, there is no multi-tenancy infrastructure, connection pooling, or admin interface.

No test coverage: The codebase has no automated test suite. Unit tests for cosine similarity, integration tests for the upload pipeline, and end-to-end tests for the RAG pipeline would be essential before production deployment.

Whisper API cost for audio/video: Unlike OCR (Tesseract.js — free), audio and video transcription through OpenAI Whisper incurs API costs at scale. Users with large audio/video libraries may face non-trivial transcription expenses.

CHAPTER 7: CONCLUSION AND FUTURE WORK

7.1 Conclusion

Knowledge Vault demonstrates that a focused undergraduate project can implement a technically rigorous, production-quality AI system that integrates multiple advanced techniques — Retrieval-Augmented Generation, semantic vector embeddings, knowledge graph construction, supervised machine learning classification, automated insight generation, and quantitative evaluation — into a cohesive product that solves a genuine problem for its target users.

The project's central technical claim — that grounding LLM responses in personal document content through RAG retrieval produces higher-quality, more faithful answers than either pure LLM generation or keyword-based retrieval — is empirically validated by the groundedness score of 8.5/10. The RAG pipeline reliably retrieves relevant documents (retrieval accuracy: 8.0/10) and generates answers that are well-supported by those documents, achieving an overall quality score of 7.75/10.

The ML classification experiment provides a rigorous methodological contribution: a comparative evaluation of four classifiers on a curated 11-class academic text classification dataset, producing results (SVM: 72.41%, Naive Bayes: 68.97%, LR: 65.52%, Random Forest: 48.28%) consistent with established literature benchmarks. The per-class analysis reveals specific failure patterns predicted by the literature.

The multimodal document ingestion pipeline — combining pdf-parse for PDFs, Tesseract.js OCR for images, OpenAI Whisper for audio, and ffmpeg + Whisper for video — represents a comprehensive content extraction solution that transforms documents of any format into semantically searchable knowledge. The knowledge graph's three-layer matching algorithm solves the vocabulary mismatch problem through a domain knowledge approach that is faster, cheaper, and more controllable than LLM-based entity extraction.

The most important design lesson embedded in this project is that AI systems must be evaluable, not just functional. Every module produces measurable outcomes: ML classification has accuracy and F1 scores; RAG retrieval has groundedness and relevance scores; the knowledge graph has defined

connection types and edge labelling. This commitment to measurement distinguishes engineering from tinkering and makes the system's claims defensible.

7.2 Future Work

Priority 1: Gemini Retry and Fallback

A robust Gemini utility with exponential backoff and model fallback would eliminate the most common user-facing error — intermittent 503 overload responses from the free tier during peak demand:

```
async function generateWithRetry(prompt, options = {}) {
  const models = ['gemini-2.5-flash', 'gemini-1.5-flash'];
  for (const modelName of models) {
    for (let attempt = 1; attempt <= 3; attempt++) {
      try {
        const model = ai.models;
        return await model.generateContent({ model: modelName, ... });
      } catch (err) {
        if (err.status === 503 || err.status === 429) {
          await delay(5000 * Math.pow(2, attempt - 1));
        } else { throw err; }
      }
    }
  }
}
```

Priority 2: Local Whisper Model (Cost Reduction)

Replacing the OpenAI Whisper API with the open-source whisper.cpp (C++ implementation callable from Node.js via `child_process`) would eliminate transcription API costs entirely. whisper.cpp achieves near-identical accuracy to the cloud API, supports all audio formats, and can run on CPU. For users with large audio/video libraries, this would be the most impactful cost reduction.

Priority 3: Vector Database Integration

For libraries exceeding 500 documents, FAISS integration into the Flask ML microservice would replace in-memory cosine similarity. FAISS scales to millions of vectors with sub-millisecond response times:

```
import faiss
index = faiss.IndexFlatIP(768)
index.add(np.array(embeddings))
```

```
scores, indices = index.search(np.array([query_embedding]), k=4)
```

Priority 4: Active Learning for ML Classification

A "Fix Classification" button on the document card would allow users to provide the correct label when the classifier misclassifies a document. Corrected samples accumulate in a separate MongoDB collection. A periodic retraining job reads the original training data combined with all corrections and retrains all four classifiers, automatically selecting the best performer. This creates a continuously self-improving classification system.

Priority 5: GraphRAG Integration

The current RAG pipeline retrieves documents by query similarity. GraphRAG augments this with knowledge graph traversal: starting from the most similar documents, follow concept edges to retrieve conceptually related documents even if they are not directly similar to the query text. This would particularly benefit queries requiring cross-document synthesis, such as "explain the relationship between memory management and process scheduling."

Priority 6: Multi-User Collaboration

Shared knowledge bases between users would enable study groups to pool their document libraries, with role-based access control distinguishing owner, editor, and viewer permissions. The backend's JWT authentication and document-level user filtering provide a clean foundation for this extension.

Priority 7: Study Analytics Dashboard

Tracking user interactions — which documents are accessed, which queries are asked, which knowledge gaps are followed up — would enable a personalised study analytics dashboard showing knowledge base coverage over time, most queried topics, unanswered queries indicating knowledge gaps, and study streaks for habit formation.

Priority 8: Mobile Application

A React Native application sharing the existing REST API would enable document capture (photograph handwritten notes with phone camera → automatic Tesseract OCR → indexed in Knowledge Vault), on-the-go RAG queries, and push notifications for spaced repetition reminders. The backend's stateless JWT authentication and JSON APIs already make mobile integration straightforward.

CHAPTER 8: REFERENCES

- [1] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, "Retrieval-Augmented Generation for Large Language Models: A Survey," arXiv preprint arXiv:2312.10997, 2024.
- [2] S. Gupta, R. Ranjan, and S. N. Singh, "A Comprehensive Survey of Retrieval-Augmented Generation (RAG): Evolution, Current Landscape and Future Directions," arXiv preprint arXiv:2410.12837, 2024.
- [3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [4] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," arXiv preprint arXiv:2309.15217, 2023.
- [5] M. Menschikov, A. Ivanov, P. Petrov, and colleagues, "PersonalAI: A Systematic Comparison of Knowledge Graph Storage and Retrieval Approaches for Personalized LLM Agents," arXiv preprint arXiv:2506.17001, 2025.
- [6] S. Minaee, T. Mikolov, N. Nikzad, M. Chenaghlu, R. Socher, X. Amatriain, and J. Gao, "Large Language Models: A Survey," arXiv preprint arXiv:2402.06196, 2025.
- [7] Y. Zhang, Z. Liang, L. Li, and colleagues, "A Comprehensive Survey on Automatic Text Summarization with Exploration of LLM-Based Methods," arXiv preprint arXiv:2403.02901, 2025.
- [8] J. Lee, Z. Xu, J. Chen, and Google DeepMind team, "Gemini Embedding: Generalizable Embeddings from Gemini," arXiv preprint arXiv:2503.07891, 2025.
- [9] H. Cao, X. Wang, and colleagues, "Recent Advances in Universal Text Embeddings: A Comprehensive Review of Top-Performing Methods on the MTEB Benchmark," arXiv preprint arXiv:2406.01607, 2024.
- [10] S. S. Monir and D. Zhao, "VectorSearch: Enhancing Document Retrieval with Semantic Embeddings and Optimized Search," arXiv preprint arXiv:2409.17383, 2024.

- [11] A. Patel and colleagues, "VectorSearch: An Efficient Multi-Vector Embedding Framework for Scalable Semantic Retrieval," 2024.
- [12] Y. Zhang, Z. Zhang, W. Chen, H. Chen, and N. Zhang, "LLMs for Knowledge Graph Construction and Reasoning: Recent Capabilities and Future Opportunities," arXiv preprint arXiv:2305.13168, 2024.
- [13] X. Li, Y. Liu, X. Wang, and colleagues, "LLM-Empowered Knowledge Graph Construction: A Survey," arXiv preprint arXiv:2510.20345, 2024.
- [14] H. Wang, Y. Chen, and colleagues, "Towards Practical GraphRAG: Efficient Knowledge Graph Construction and Hybrid Retrieval at Scale," arXiv preprint arXiv:2507.03226, 2024.
- [15] Z. Chen, Y. Liu, J. Wang, and colleagues, "NLP-AKG: Few-Shot Construction of NLP Academic Knowledge Graph Based on LLM," arXiv preprint arXiv:2502.14192, 2024.
- [16] S. Barnett, S. Kurniawan, S. Thudumu, Z. Brannelly, and M. Abdelrazek, "Seven Failure Points When Engineering a Retrieval Augmented Generation System," arXiv preprint arXiv:2401.05856, 2024.
- [17] K. Kowsari, K. Jafari Meimandi, M. Heidarysafa, S. Mendu, L. E. Barnes, and D. E. Brown, "Text Classification Algorithms: A Survey," *Information*, vol. 10, no. 4, p. 150, 2019. arXiv preprint arXiv:1904.08067.
- [18] L. Galke and A. Scherp, "Are We Really Making Much Progress in Text Classification? A Comparative Review," arXiv preprint arXiv:2204.03954, 2024.
- [19] N. Ibrahim, S. Aboulela, A. Ibrahim, and R. Kashef, "Automated Research Article Classification and Recommendation Using NLP and Machine Learning," arXiv preprint arXiv:2510.05495, 2025.
- [20] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Schmidtke, and J. Larson, "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," Microsoft Research Technical Report, 2024.

APPENDIX A: API ENDPOINT REFERENCE

Method	Endpoint	Auth	Request Body	Response	Description
POST	/api/auth/register	No	{name, email, password}	{token, user}	Register new user
POST	/api/auth/login	No	{email, password}	{token, user}	Login and receive JWT
GET	/api/auth/me	Yes	—	{user}	Get current user info
POST	/api/documents/upload	Yes	multipart: file	{document}	Upload and process file
GET	/api/documents	Yes	?type=&search=	[{documents}]	List documents with filters
GET	/api/documents/:id	Yes	—	{document}	Get single document
DELETE	/api/documents/:id	Yes	—	{message}	Delete document and file
GET	/api/documents/stats	Yes	—	{pdf,image,audio,video,total}	File type counts
POST	/api/documents/embed-all	Yes	—	{count}	Re-embed all documents
POST	/api/chat	Yes	{message, conversationHistory}	{response, retrievedDocs}	RAG chat query
POST	/api/insights/generate	Yes	—	{overview, themes, gaps, studySequence}	Generate insights
POST	/api/eval/run	Yes	—	{results, avgScores, evalId}	Run evaluation (3 Gemini calls)
POST	/api/eval/feedback	Yes	{question, response, rating}	{message}	Save chat feedback
GET	/api/eval/history	Yes	—	[{runs}]	Past evaluation runs
GET	/api/ml/health	No	—	{status, model_loaded}	ML server status check

GET	/api/ml/results	No	—	{classifiers, per_class, confusion_matrix}	Training results
POST	/api/ml/predict	Yes	{text}	{subject, confidence, all_scores}	Classify text input
POST	/api/ml/classify-all	Yes	—	{classified, total, results}	Bulk classify documents
POST	/api/ml/retrain	Yes	—	{status, output}	Trigger model retraining

Table A.1: Complete REST API Documentation

APPENDIX B: ML TRAINING DATASET STATISTICS

Subject Class	Training Samples	Test Samples	Total	% of Dataset
Machine Learning	14	4	18	12.5%
OOP	12	3	15	10.4%
Data Structures	12	3	15	10.4%
Algorithms	12	3	15	10.4%
DBMS	12	3	15	10.4%
Operating Systems	12	3	15	10.4%
Computer Networks	12	3	15	10.4%
Web Development	10	2	12	8.3%
Mathematics	9	3	12	8.3%
Computer Science	5	1	6	4.2%
Software Engineering	5	1	6	4.2%
Total	115	29	144	100%

Table B.1: Training Dataset Distribution by Class

APPENDIX C: SYSTEM REQUIREMENTS

Component	Minimum Version	Recommended	Notes
Node.js	18.x	20.x LTS	Required for native fetch API and ESM
Python	3.9	3.11	scikit-learn 1.8 requires Python 3.9+
MongoDB	6.0	7.0	Local or Atlas free tier
npm	9.x	10.x	Included with Node.js
pip	23.x	Latest	Use --break-system-packages if needed
ffmpeg	4.x	6.x	Required for video audio extraction

Table C.1: Software Requirements

Package	Purpose
scikit-learn 1.8.x	ML classifiers and TF-IDF vectorizer
flask 2.x	REST API server
flask-cors 4.x	CORS for Node.js cross-origin calls
pandas 2.x	Training CSV loading
numpy 1.x	Numerical operations (softmax)

Table C.2: Python Package Requirements

Package	Purpose
@google/generative-ai	Gemini API client (generation + embeddings)
openai	Whisper API client (audio/video transcription)
tesseract.js	OCR engine for image text extraction
mongoose	MongoDB ODM with schema validation
multer	Multipart file upload middleware
pdf-parse	PDF text layer extraction
jsonwebtoken	JWT generation and verification
bcryptjs	Password hashing (bcrypt algorithm)
express	REST API framework

cors	Cross-origin request handling
dotenv	Environment variable loading
nodemon (dev)	Development auto-restart

Table C.3: Node.js Package Requirements

Resource	Minimum	Recommended
RAM	4 GB	8 GB
Storage	2 GB	10 GB
Internet	Required	Stable broadband (Gemini + Whisper API)
OS	Windows 10, macOS 11, Ubuntu 20.04	Any current OS

Hardware Recommendations

APPENDIX D: DIRECTORY STRUCTURE

```
knowledge-vault/
├── backend/
│   ├── middleware/
│   │   └── auth.js
│   ├── models/
│   │   ├── User.js
│   │   ├── Document.js
│   │   └── EvalResult.js
│   ├── routes/
│   │   ├── auth.js
│   │   ├── documents.js
│   │   ├── chat.js
│   │   ├── insights.js
│   │   ├── eval.js
│   │   └── ml.js
│   ├── utils/
│   │   └── embeddings.js
│   ├── uploads/
│   ├── server.js
│   ├── package.json
│   └── .env
├── frontend/
│   ├── src/
│   │   ├── pages/
│   │   │   ├── Login.jsx
│   │   │   ├── Register.jsx
│   │   │   ├── Dashboard.jsx
│   │   │   ├── Chatbot.jsx
│   │   │   ├── KnowledgeGraph.jsx
│   │   │   ├── Insights.jsx
│   │   │   ├── Eval.jsx
│   │   │   └── MLClassifier.jsx
│   │   ├── App.jsx
│   │   ├── main.jsx
│   │   └── index.css
│   ├── index.html
│   ├── vite.config.js
│   └── package.json
└── ml/
```

```
|— training_data.csv
|— train.py
|— predict.py
|— model.pkl
└— results.json
```